

SQLP 과목 III 튜닝 스타일 모의문제 · 9회차

SQL 자격검정 실전문제 3장 스타일 · 4지선다 · 총 20문항 · 원본 창작

1

대량 데이터를 읽고 쓸 때 I/O를 줄이는 세 축 — 인접 블록을 한 Call에 묶는 Multiblock I/O, 버퍼 캐시(SGA)를 우회하는 Direct Path I/O, 그리고 읽는 블록 수 자체를 줄이는 액세스 경로 최적화 — 에 대한 설명으로 가장 적절하지 않은 것은?

- ① Multiblock I/O는 Full Table Scan처럼 인접한 여러 블록을 한 번의 I/O Call로 묶어 읽어 들이므로, 같은 블록 수를 읽더라도 블록을 하나씩 집어 오는 방식보다 I/O Call 횟수를 줄여 준다.
- ② Direct Path Read는 읽어 들인 블록을 SGA 버퍼 캐시에 올리지 않고 서버 프로세스의 PGA로 곧바로 가져오지만, 그 세그먼트에 아직 데이터파일에 반영되지 않은 더티 블록이 캐시에 남아 있으면 그것을 먼저 내려쓰는 세그먼트 체크포인트가 선행되어야 한다.
- ③ I/O 효율화의 뿌리는 애초에 필요 없는 블록을 읽지 않도록 스캔 범위와 액세스 경로를 다듬는 데 있으며, Multiblock I/O나 Direct Path Read는 이미 읽어야 할 블록을 더 싸게 읽어 주는 수단일 뿐 읽어야 할 블록 수 자체를 줄여 주지는 못한다.
- ④ Multiblock I/O와 Direct Path Read는 여러 블록을 한 묶음으로 처리하므로 같은 데이터를 읽을 때 방문해야 할 논리적 블록 수(논리적 I/O) 자체가 줄어들며, 이렇게 논리적 I/O를 깎아 주는 것이 두 기법이 주는 I/O 효율화의 본질이다.

2

SGA와 프로세스가 데이터 변경을 나눠 처리하는 구조 — 사용자의 SQL을 받아 실행하는 서버 프로세스, 그리고 각자 정해진 일을 맡는 백그라운드 프로세스(DBWR·LGWR·CKPT) — 에서 각 주체의 역할로 가장 적절한 것은?

- ① 서버 프로세스는 사용자의 SQL을 파싱·실행해 필요한 블록을 DB 버퍼 캐시로 올려 처리하는 한편, 자신이 변경한 더티 버퍼를 데이터파일에 내려쓰는 일까지 직접 맡아 DBWR가 개입하지 않아도 블록 기록이 이루어진다.
- ② CKPT(체크포인트 프로세스)는 이름 그대로 체크포인트가 발생하면 DB 버퍼 캐시의 더티 버퍼를 데이터파일에 직접 써 내려, 변경 블록을 디스크에 반영하는 실제 쓰기를 도맡는다.
- ③ LGWR는 커밋이 떨어지거나 로그 버퍼가 일정 수준 차오르면 Redo 로그 버퍼의 변경 벡터를 Redo 로그 파일에 기록하며, 커밋은 이 Redo 기록이 끝나야 완료되는 것이지 변경 블록이 데이터파일에 반영되는 것과는 분리되어 있다.
- ④ DBWR는 더티 버퍼를 데이터파일에 기록하기에 앞서 그 변경을 담은 Redo를 로그 파일에 자신이 직접 먼저 남기는 로그 선기록(WAL)까지 수행하므로, LGWR가 동작하지 않아도 복구에 필요한 Redo가 확보된다.

특허출원 심사관 자동배정 배치가 접수된 출원 50만 건을 순회하며 건마다 심사관 배정 UPDATE 1건씩(총 50만 건)을 실행한다. 개발자가 출원번호·기술분류코드 같은 조건값을 바인드 변수가 아니라 문자열로 직접 이어 붙여(리터럴) SQL을 만들었기 때문에, 실행마다 SQL 텍스트가 조금씩 달라진다. 8개의 배정 세션이 동시에 돌아 응답이 49초로 늘고 대기 이벤트 상위에 library cache: mutex X·cursor: pin S wait on X·shared pool latch가 잡혔다. 아래는 이 배치 세션의 비재귀·재귀 statement를 나눠 집계한 TKPROF이다. 지연의 원인을 직접 지목한 것으로 옳은 것은? (단, 옵티마이저 통계는 최신이고 Shared Pool은 여유가 충분해 에이징으로 인한 강제 재파싱은 없으며, 각 실행은 조건값만 다르고 UPDATE 구조는 동일하다.)

아 래							
OVERALL TOTALS FOR ALL NON-RECURSIVE STATEMENTS							
Call	Count	CPU Time	Elapsed	Disk	Query	Current	Rows
Parse	500000	40.000	42.000	0	0	0	0
Execute	500000	6.000	7.000	0	1000000	600000	500000
Fetch	0	0.000	0.000	0	0	0	0
Total	1000000	46.000	49.000	0	1000000	600000	500000
Misses in library cache during parse: 499000							
OVERALL TOTALS FOR ALL RECURSIVE STATEMENTS							
Call	Count	CPU Time	Elapsed	Disk	Query	Current	Rows
Parse	2000000	8.000	8.000	0	0	0	0
Execute	2000000	10.000	10.000	0	8000000	0	0
Fetch	3000000	5.000	5.000	0	12000000	0	3000000
Total	7000000	23.000	23.000	0	20000000	0	3000000
Misses in library cache during parse: 40							

- ① 비재귀 Parse 콜 500,000과 Execute 콜 500,000이 정확히 1:1이고 Parse에 잡힌 Elapsed 42.0초가 Total 49.0초의 85.7%를 차지하므로, 실행마다 커서를 놓쳤다가 다시 찾는 소프트파싱 콜 폭주가 병목이다. session_cached_cursors를 올리고 커서를 닫지 않고 홀드해 Parse 콜을 없애면 라이브러리 캐시 mutex 대기과 49초가 함께 줄어든다.
- ② 재귀 statement의 Query 논리 읽기가 Execute 8,000,000과 Fetch 12,000,000을 합쳐 20,000,000으로 비재귀 1,000,000의 20배에 달해, 데이터 디크너리 블록을 과도하게 읽는 것이 병목이다. DB 버퍼 캐시를 키워 이 논리 읽기를 캐시 적중으로 돌리면 재귀 Total에 잡힌 23.0초가 회수되어 파싱 지연이 사라진다.
- ③ Execute 콜 500,000이 Query 1,000,000·Current 600,000 합계 1,600,000블록을 만져 갱신 1건당 3.2블록을 읽고 Execute Elapsed 7.0초를 쓴 것이 실제 갱신 작업의 골격이므로, UPDATE 실행계획을 다시 짜 건당 블록 수를 줄이면 실행 속이 가벼워져 49초가 해소되고 재귀 Call 700만도 함께 줄어든다.
- ④ 라이브러리 캐시 미스가 499,000건으로 비재귀 Parse 50만 콜의 99.8%에 달해 사실상 매 실행이 하드파싱이고, 그 하드파싱이 데이터 디크너리를 뒤지며 재귀 Call 700만(비재귀의 7배)을 유발한 것이 원인이다. 리터럴을 바인드 변수로 바꿔 커서를 공유시키면 하드파싱이 소프트파싱으로 바뀌어 49초가 해소된다.

4

오라클에서 옵티마이저가 추정한 예상 실행계획과, 문장을 실제로 돌려 얻는 실측 행 수가 담긴 실제 실행계획을 각각 어떻게 얻는지 — gather_plan_statistics 힌트, DBMS_XPLAN.DISPLAY_CURSOR, 실시간 SQL 모니터링 — 에 대한 설명으로 가장 적절한 것은?

- ① gather_plan_statistics 힌트를 넣은 문장은 실행하지 않아도 옵티마이저가 예상 행 수와 함께 실측 행 수를 미리 계산해 두므로, 문장을 수행하지 않고 DBMS_XPLAN.DISPLAY_CURSOR로 예상치와 실측치를 나란히 비교할 수 있다.
- ② gather_plan_statistics 힌트를 주거나 STATISTICS_LEVEL을 ALL로 올린 뒤 문장을 실제로 수행하고, DBMS_XPLAN.DISPLAY_CURSOR(format=>'ALLSTATS LAST')로 조회하면 각 단계의 예상 행 수와 실측 행 수를 함께 볼 수 있어 추정 오차가 큰 단계를 짚어낼 수 있다.
- ③ V\$SQL_PLAN에 담기는 계획은 EXPLAIN PLAN이 PLAN_TABLE에 남기는 예상 계획과 같은 것이라, 문장을 수행하지 않아도 라이브러리 캐시에 미리 만들어져 실측 없이 그대로 조회된다.
- ④ 실시간 SQL 모니터링 리포트(DBMS_SQLTUNE.REPORT_SQL_MONITOR)는 문장을 수행하기 전 옵티마이저의 추정만으로 만들어지는 예상 계획 리포트라, 각 단계의 실측 행 수나 소요 시간 같은 실측 정보는 담기지 않는다.

5

축산물 이력 시스템에서 등급판정이력(약 2억 건, 테이블 약 999,000 블록)을 도축장(약 300건)과 조인해 도축장·축종별 집계 200만 행을 뽑는 SQL의 TKPROF이다. 등급판정이력 Full Scan은 멀티블록이라 금방 끝날 것으로 봤는데 응답이 60초를 넘겨 느리다는 신고가 들어왔다. 아래 트레이스에 대한 해석으로 가장 적절한 것은? (단, 옵티마이저 통계는 최신이고 CPU 자원의 외부 경합은 없으며, 두 테이블에 인덱스가 없어 Full Scan + Hash Join으로만 수행되고 등급판정이력은 대부분 버퍼 캐시에 없어 물리 I/O가 크게 발생했다.)

아 래							
Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.010	0.010	0	0	0	0
Execute	1	0.000	0.000	0	0	0	0
Fetch	2000	10.000	60.000	700000	1000000	0	2000000
Total	2002	10.010	60.010	700000	1000000	0	2000000
Rows	Row Source Operation						
20000000	HASH GROUP BY (cr=1000000 pr=0 pw=0 time=59500000 us)						
200000000	HASH JOIN (cr=1000000 pr=700000 pw=0 time=57000000 us)						
300	TABLE ACCESS FULL 도축장 (cr=1000 pr=0 pw=0 time=2000 us)						
2000000000	TABLE ACCESS FULL 등급판정이력 (cr=999000 pr=700000 pw=0 time=50000000 us)						

- ① Fetch 콜 2,000회에 CPU Time 10초가 몰려 있고, HASH JOIN이 2억 행을 뽑아 내고 HASH GROUP BY가 그 2억 행을 200만 행으로 줄이는 해시 연산은 둘 다 CPU를 태우므로 이 SQL은 CPU 바운드다. Elapsed 60초는 단일 프로세스가 그 CPU 작업을 혼자 감당한 결과이니, 병렬도(DOP)를 높여 CPU를 여러 슬레이브에 나눠 주면 60초가 개선된다.
- ② BCHR이 30.0%로 낮고, 물리 읽기 700,000이 모두 등급판정이력 Full Scan에 붙어 있으며(pw=0이라 spill 없음) 그 노드 time이 50초다. 병목은 논리 읽기나 집계가 아니라 캐시에 없는 큰 테이블을 디스크에서 읽는 물리 I/O이므로, 스캔 대상을 좁히거나(파티션 프루닝) 캐시 적중을 높여 물리 읽기를 줄여야 한다.
- ③ cr 1,000,000 중 등급판정이력 Full Scan이 999,000(99.9%)이고 도축장은 1,000뿐이라 논리 읽기를 이 한 노드가 지배하며, 그 노드 time도 50초에 이른다. 병목은 이 노드가 훑는 999,000블록의 논리 읽기 자체이고, Disk 700,000의 물리 읽기는 그에 비례해 따라붙는 부차적 수치이므로 캐시 적중을 손보는 처방은 결가지다.
- ④ HASH GROUP BY 노드의 cr이 1,000,000으로 HASH JOIN과 같고 time도 59.5초로 플랜에서 가장 길며, 200만 그룹을 해시 영역에 못 담아 temp로 흘린 spill이 물리 읽기 700,000의 정체다. PGA(해시/정렬 영역)를 키워 200만 그룹을 메모리 안에서 끝내면 이 spill이 사라져 Fetch Elapsed 60초도 함께 사라진다.

전자여권 발급 관리 시스템의 발급신청 테이블(약 880만 건)에는 결합 인덱스 신청_IX = (발급기관코드, 접수일시)가 있다. 담당 발급기관명을 함께 얻기 위해 발급기관 테이블(기본키 발급기관_PK = 발급기관코드)과 조인하는 다음 SQL의 실행계획이다. 이 실행계획에 대한 설명으로 가장 적절한 것은?

아 래

```
SELECT s.신청번호, s.접수일시, s.신청인성명, o.발급기관명
FROM 발급신청 s, 발급기관 o
WHERE s.발급기관코드 = 'A21'
      AND s.접수일시 >= DATE '2026-06-01'
      AND s.접수일시 < DATE '2026-07-01'
      AND s.긴급여부 = 'Y'
      AND o.발급기관코드 = s.발급기관코드;
```

Id	Pid	Operation	(Cost	Card	Bytes)
0	-	SELECT STATEMENT	(356	2800	159600)
1	0	NESTED LOOPS	(356	2800	159600)
2	1	TABLE ACCESS BY INDEX ROWID 발급신청	(350	2800	100800)
3	2	INDEX RANGE SCAN 신청_IX	(26	9500	)
4	1	TABLE ACCESS BY INDEX ROWID 발급기관	(1	1	21)
5	4	INDEX UNIQUE SCAN 발급기관_PK	(0	1	)

- ① 발급기관코드(등치)와 접수일시(범위)까지가 신청_IX의 인덱스 액세스 조건으로 INDEX RANGE SCAN에서 처리(Card 9,500)되고, 인덱스에 없는 긴급여부는 TABLE ACCESS BY INDEX ROWID 단계 필터로 걸려져 Card가 2,800으로 줄어든다.
- ② WHERE의 세 조건(발급기관코드 등치·접수일시 범위·긴급여부 등치)이 모두 신청_IX의 인덱스 액세스 조건으로 Id 3 INDEX RANGE SCAN 단계에서 함께 처리되어 스캔 시작·종료점을 결정하며, 그렇게 좁혀 얻은 결과가 Card 9,500이다.
- ③ Id 3 INDEX RANGE SCAN이 반환한 Card 9,500이 곧 이 SQL의 최종 결과 건수이며, Id 2 TABLE ACCESS BY INDEX ROWID는 ROWID로 행을 꺼내 올 뿐 건수 감소를 일으키지 않는다. Id 2와 Id 3의 Cost 차이(350 대 26)는 테이블 블록 방문 비용일 뿐이다.
- ④ 접수일시가 범위(>=, <) 조건이므로 선두 칼럼 발급기관코드 = 'A21'의 등치도 인덱스 액세스 조건이 되지 못하고, 신청_IX를 전 구간 훑은 뒤 스캔 후 필터로만 처리된다. Id 3의 Cost 26이 그 전 구간 스캔 비용이며, Card 9,500은 필터 통과 건수다.

전기차 배터리 셀 검사 시스템에서 셀검사이력(약 2,000만 건, 테이블 총 블록 약 200,000)을 배터리모델코드 인덱스로 검색해 특정 모델의 검사 2,000,000행(전체의 10%)을 인덱스 순서로 읽어 내려받는 SQL의 TKPROF이다. 인덱스로 잘 타는데도 응답이 70초를 넘긴다. 개발자는 "인덱스를 탔으니 최선"이라 보지만, 아래 트레이스를 클러스터링 팩터와 인덱스 손익분기점 두 지표로 진단할 때 실제 병목은 무엇인가? (단, 옵티마이저 통계는 최신이고, 애플리케이션은 이미 배열 인출을 사용하며 인덱스는 배터리모델코드 단일 컬럼으로 구성돼 있다.)

아래							
Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.003	0.003	0	0	0	0
Execute	1	0.000	0.000	0	0	0	0
Fetch	2001	18.000	70.000	250000	1803000	0	2000000
Total	2003	18.003	70.003	250000	1803000	0	2000000
Rows	Row Source Operation						
2000000	TABLE ACCESS BY INDEX ROWID 셀검사이력 (cr=1803000 pr=250000 pw=0 time=66000000 us)						
2000000	INDEX RANGE SCAN 셀검사이력_모델_IDX (cr=3000 pr=800 pw=0 time=1200000 us)						

- ① 물리 읽기가 테이블 노드 pr 250,000·인덱스 노드 pr 800으로 사실상 전부 테이블 쪽에 붙어 있고 그 노드 time이 66초이므로, 70초 대기를 만든 근본 원인은 이 250,000블록의 디스크 읽기다. DB 버퍼 캐시를 키워 250,000블록을 캐시 적중으로 돌리면 랜덤 액세스가 사라져 66초가 회수된다.
- ② 논리 읽기 cr 1,803,000이 반환 행 2,000,000에 육박하고 Fetch 콜 2,001회로 나누면 쿨당 약 900블록이라 이 논리 읽기 과다가 병목이다. 인덱스를 리빌드해 흩어진 리프를 조밀하게 다지고 단편화를 걷어 내 인덱스 스캔의 cr을 줄이면 66초짜리 테이블 노드까지 가버려져 70초가 개선된다. Disk 250,000도 그 결과다.
- ③ 인덱스 경우 테이블 랜덤 블록이 1,800,000이라 CF 밀도가 0.90이고, 이는 Full Scan이 읽을 테이블 총 블록 200,000의 9배다. 10% 선택도에서 인덱스가 손익분기점을 넘어 손해이므로 병목은 인덱스가 아니라 테이블 랜덤 액세스이며, Full Scan 전환이나 CF 개선·커버링으로 테이블 액세스를 없애야 한다.
- ④ CPU Time 18초가 Elapsed 70초의 빠대이고, 인덱스 키를 비교하며 2,000,000행을 ROWID로 하나씩 꺼내는 작업이 CPU를 태우므로 이 SQL은 CPU 바운드가. 단일 프로세스가 그 18초를 혼자 감당하느라 70초로 늘어난 것이니, 병렬도(DOP)를 높여 CPU를 나눠 주면 70초가 개선된다. 인덱스 노드 time이 1.2초뿐인 것도 인덱스는 멀쩡하다는 뜻이다.

인덱스만 읽고 테이블을 되짚지 않는 인덱스 전용 스캔(커버링)과, 인덱스 전체를 읽는 두 방식(Index Full Scan·Index Fast Full Scan)의 성립 조건과 효과에 대한 설명으로 가장 적절하지 않은 것은?

- ① 인덱스 전용 스캔(커버링)은 쿼리가 참조하는 컬럼이 인덱스에 모두 들어 있을 때 성립하는데, 이때 옵티마이저는 리프에서 얻은 각 ROWID로 테이블 블록을 한 번씩 방문해 컬럼 값을 채우므로 오히려 테이블 Random 액세스가 늘어난다.
- ② Index Full Scan은 선두 컬럼에 조건이 없어 Range Scan을 못 쓰더라도, 조회에 필요한 컬럼이 인덱스에 다 있으면 리프를 연결 순서대로 한 블록씩 훑어 테이블을 건드리지 않고 결과를 낼 수 있으며, 인덱스가 테이블보다 작으면 Full Table Scan보다 읽을 블록이 적어 유리하다.
- ③ Index Fast Full Scan은 인덱스 세그먼트의 블록(브랜치·리프)을 물리적 저장 순서대로 Multiblock I/O로 읽어 대량 처리·병렬에 유리하지만, 인덱스에 저장되지 않는 NULL 행을 놓칠 수 있어 인덱스 컬럼 중 하나 이상이 NOT NULL이어야 테이블 대신 쓸 수 있다.
- ④ 인덱스 전용 스캔이 가능한지는 스캔 방식과 별개의 문제라, 쿼리에 필요한 컬럼이 인덱스에 다 들어 있으면 Index Range Scan이든 Index Full Scan이든 그 방식과 무관하게 테이블 액세스(TABLE ACCESS BY INDEX ROWID)를 생략할 수 있다.

9

기본키(PK)·유니크 제약과 그것을 뒷받침하는 인덱스의 관계, 그리고 제약과 인덱스가 별개 오브젝트라는 점에 대한 설명으로 가장 적절하지 않은 것은?

- ① PK 제약을 만들면 오라클은 그 제약을 강제할 인덱스를 자동으로 생성하는데, 같은 컬럼 구성의 유니크 인덱스가 이미 있으면 새로 만들지 않고 그 인덱스를 재사용해 제약을 뒷받침한다.
- ② PK·유니크 제약을 꼭 유니크 인덱스로만 뒷받침해야 하는 것은 아니며, 제약을 지연(DEFERRABLE)으로 두거나 그 제약 컬럼들을 선두에 포함한 비유니크 결합 인덱스가 있으면 비유니크 인덱스로도 제약을 지원할 수 있다.
- ③ 제약과 인덱스는 서로 독립된 오브젝트이므로, PK 제약을 삭제하면 그 제약을 뒷받침하던 인덱스는 생성 경위와 상관없이 항상 그대로 남아 이후 조회에 계속 활용된다.
- ④ PK·유니크 제약은 값의 중복을 막는 무결성 규칙이고 인덱스는 그 규칙을 값 중복 여부로 빠르게 검사하게 해 주는 수단이라, 행을 넣을 때마다 제약 컬럼 값이 인덱스에 이미 있는지 확인해 중복이면 거부한다.

10

병역판정검사 관리 시스템에서, 특정 지방병무청의 당월 검사대상자만 뽑아 각자의 판정결과를 함께 조회한다. 구동 집합인 검사대상자 테이블(당월 대상 약 3만 건)을 인덱스 범위로 훑고, 각 대상자마다 판정결과 테이블(약 5,200만 건)을 기본키로 한 건씩 찾아 붙이는 다음 실행계획이다. 이 플랜의 조인 처리 방식에 대한 설명으로 가장 적절하지 않은 것은?

아 래

```
SELECT t.대상자ID, t.성명, t.검사에정일, r.판정구분, r.신체등급
FROM 검사대상자 t, 판정결과 r
WHERE t.관할병무청 = '31'
      AND t.검사에정월 = '202607'
      AND r.대상자ID = t.대상자ID;
```

Id	Operation	Name
0	SELECT STATEMENT	
1	NESTED LOOPS	
2	TABLE ACCESS BY INDEX ROWID	검사대상자
* 3	INDEX RANGE SCAN	대상자_IX
4	TABLE ACCESS BY INDEX ROWID	판정결과
* 5	INDEX UNIQUE SCAN	판정결과_PK

- ① 이 플랜은 구동 집합 검사대상자를 먼저 해시 테이블로 적재한 뒤 판정결과를 스캔하며 탐침하는 해시 조인이며, 판정결과를 인덱스로 한 건씩 반복 탐색하지는 않는다.
- ② Id 1이 NESTED LOOPS이므로, 구동 테이블 검사대상자(Id 2~3)에서 나온 각 행마다 판정결과(Id 4~5)를 한 번씩 파고드는 NL 조인이다.
- ③ 안쪽 판정결과는 판정결과_PK를 통한 INDEX UNIQUE SCAN으로 대상자ID당 한 건씩 접근하므로, 판정결과 쪽 조인 칼럼에 인덱스가 있어야 이 플랜이 성립한다.
- ④ 조인 순서는 검사대상자 → 판정결과이며, 구동 집합이 3만 건 수준으로 작아 판정결과를 반복 탐침해도 되는 상황에서 NL 조인이 선택된 형태다.

11

해시 조인(Hash Join)이 두 입력을 읽는 횟수와 조인 키 분포에 대한 설명으로 가장 적절한 것은?

- ① 해시 조인은 Build Input을 한 번 읽어 해시 테이블을 만든 뒤 Probe Input을 한 번 훑어 조회하므로, 두 입력을 각각 한 번씩만 읽고 끝난다. 드라이빙 건마다 상대 집합을 되읽는 반복 스캔이 없어 대량 집합끼리의 조인에서 이 점이 이점이 된다.
- ② 해시 조인은 조인 키를 해시 함수로 버킷에 고르게 흩어 담고 Probe Input의 각 행도 같은 함수로 자기 버킷을 찾아가므로, 조인 키 값이 특정 값에 크게 쏠려 있어도 버킷마다 행 수가 균등하게 유지된다. 따라서 Build 1회·Probe 1회라는 읽기 횟수가 곧 조회 시간을 결정한다.
- ③ 해시 조인은 Build Input을 조인 키로 미리 정렬한 뒤 그 순서대로 해시 테이블을 채우므로, Build Input 쪽에 조인 키 인덱스가 있어 정렬된 순서로 읽어 오면 정렬 단계를 건너뛰어 그 비용을 아낄 수 있다. 두 입력을 한 번씩만 읽는 이점도 정렬 비용을 없앨 때 살아난다.
- ④ 해시 조인은 버킷이 담는 키 범위를 넓게 잡으면 한 버킷 안에서 부등호·범위 비교까지 매칭할 수 있어, 등치는 물론 범위 조인에서도 Build 1회·Probe 1회로 끝내는 이점을 그대로 누리며 기본 조인 방식으로 선택된다. 정렬해 맞대는 방식이 아니어서 범위 비교에도 제약이 없다.

12

옵티마이저의 쿼리 변환을 규칙 기반(휴리스틱) 변환과 비용 기반 변환으로 나누는 관점, 그리고 특정 변환을 억제하는 힌트에 대한 설명으로 가장 적절하지 않은 것은?

- ① 조건절 Pushing이나 단순 뷰 병합처럼 적용해도 결과가 달라지지 않고 대체로 손해 볼 일이 없는 변환은, 옵티마이저가 변환 전후 비용을 건주지 않고 규칙적으로 적용하는 휴리스틱 변환으로 볼 수 있다.
- ② 쿼리 변환은 어느 것이든 변환 후 계획이 더 빨라지는 경우에만 적용되는 비용 기반 최적화이므로, 변환을 한 탓에 원래보다 계획이 느려지는 일은 원리상 생기지 않는다.
- ③ 복합 뷰 병합이나 조인 조건절 Pushdown(Join Predicate Pushdown)처럼 이득이 상황에 따라 갈리는 변환은, 옵티마이저가 변환 전후의 비용을 건줘 채택 여부를 정하는 비용 기반 변환이다.
- ④ 서브쿼리 Unnesting을 막으려면 NO_UNNEST 힌트를, 뷰 병합을 막으려면 NO_MERGE 힌트를 걸어 개별 변환을 선택적으로 억제할 수 있고, 이는 옵티마이저의 변환 판단을 사람이 덮어쓰는 수단이 된다.

13

국립전시관 관람 대시보드가 관람이력 테이블(약 4,200만 건)에서 전시·권종 조합별 관람 건수를 조회한다. 예전에는 전시코드·권종을 리터럴로 결합해 발행하는 바람에 조합 수만큼 커서가 갈리고 library cache: mutex X가 상위 대기에 잡혀, 최근 아래처럼 바인드 변수로 바꿔 발행하도록 고쳤다. 바인드 도입 뒤의 커서 공유와 라이브러리 캐시(핀·에이징·무효화) 동작에 대한 설명으로 가장 적절하지 않은 것은?

아래

```
-- 개선 후: 전시코드·권종을 바인드 변수로 발행
SELECT COUNT(*) FROM 관람이력 WHERE 전시코드 = :b1 AND 권종 = :b2;
-- :b1, :b2 에 (E027, '주간'), (E027, '야간'), (E031, '단체') ... 값만 바꿔 반복 실행

-- 대시보드 화면마다 개발자별로 텍스트가 조금씩 다르게 작성된 변형도 섞여 있다
SELECT count(*) FROM 관람이력 WHERE 전시코드 = :b1 AND 권종 = :b2; -- COUNT 대소문자·공백 상이
```

- ① 바인드 변수를 쓰면 값만 다른 실행이 같은 부모 커서를 공유하므로, 최초 실행의 하드파싱 뒤부터는 소프트파싱으로 라이브러리 캐시의 커서를 재사용해 파싱 부하와 library cache: mutex 경합이 줄어든다.
- ② 실행 중 참조되는 커서는 라이브러리 캐시에서 핀(pin)되어 에이징 대상에서 제외되지만, 사용이 끝난 커서는 공유 풀 메모리 압박 시 LRU 정책으로 에이징되어 밀려날 수 있고, 다시 필요하면 재파싱된다.
- ③ 관람이력 통계를 재수집하거나 그 테이블에 DDL을 수행하면 관련 커서가 무효화(invalidation)되어, 다음 실행 때 재파싱(재최적화)이 일어난다.
- ④ 바인드 변수를 쓰면 오라클이 SQL 텍스트의 공백·대소문자·주석을 내부적으로 정규화해 비교하므로, 바인드 SQL끼리는 COUNT와 count, 들여쓰기 차이가 있어도 하나의 부모 커서로 묶여 공유된다.

비용 기반 옵티마이저(CBO)가 산정하는 Cost(비용)의 성격과, I/O·CPU 비용 및 optimizer_index_cost_adj가 그 값에 관여하는 방식에 대한 설명으로 가장 적절한 것은?

- ① optimizer_index_cost_adj를 낮추면 인덱스 액세스의 산정 비용이 줄어 인덱스 경로가 더 자주 선택되므로, 이 파라미터를 낮추는 것은 곧 인덱스 자체의 물리적 효율(클러스터링 팩터)을 좋게 만드는 것과 같은 일이다.
- ② CBO가 매기는 Cost는 후보 계획들을 상대적으로 견주려고 예상 I/O·CPU 사용량을 하나의 수치로 환산한 추정치이며, 이 추정은 통계에서 얻은 카디널리티에서 출발하지만 그 값이 실제 경과 시간(초)과 일대일로 대응하는 것은 아니다.
- ③ 옵티마이저가 추정한 행 수(카디널리티)와 최종 Cost는 같은 것을 가리키므로, 카디널리티만 정확히 맞으면 Cost도 실제 수행 시간과 정확히 일치한다.
- ④ I/O 비용 모델과 CPU 비용 모델은 각각 디스크 읽기와 연산량을 반영하는데, CPU 비용이 큰 계획은 I/O 비용도 큰 경향이 있으므로 둘 중 한쪽만 봐도 총비용의 우열을 가릴 수 있다.

산림 생육조사 시스템의 야간 배치에 그날 수집한 임분(林分) 생육 측정치 약 6,000만 건을 스테이징(생육조사_적재)에서 조사일자 RANGE 파티션 테이블(생육조사이력)로 옮기고, 재적재 대상인 지난 분기 파티션은 통째로 비운다. 아래는 이 배치가 쓰는 DML이다. 대량 적재·삭제 튜닝에 대한 설명으로 가장 적절하지 않은 것은?

아 래

```
ALTER SESSION ENABLE PARALLEL DML;
```

```
INSERT /*+ APPEND PARALLEL(t 4) */ INTO 생육조사이력 t
SELECT * FROM 생육조사_적재;
```

— 지난 분기 재적재분은 파티션 단위로 비운다

```
ALTER TABLE 생육조사이력 TRUNCATE PARTITION p_2026q2;
```

- ① INSERT /*+ APPEND */는 Direct Path Insert로 동작해, 세그먼트 HWM 위에 새 블록을 직접 포맷해 써 넣고 버퍼 캐시를 우회하므로 대량 적재에서 재사용 블록 탐색·버퍼 경합을 줄인다.
- ② 대상 테이블이나 세션이 nologging이면 Direct Path Insert로 적재되는 데이터는 redo를 최소화(minimal logging)할 수 있어 로그 부하가 줄지만, 그 대신 미디어 복구가 필요할 때 해당 데이터를 복구하지 못할 위험이 생긴다.
- ③ ALTER TABLE ... TRUNCATE PARTITION은 파티션의 행을 한 건씩 지우며 각 행마다 Undo·Redo를 남기는 DML이라, DELETE보다 Undo 생성이 많고 도중에 ROLLBACK으로 되돌릴 수 있다.
- ④ 병렬 DML은 ALTER SESSION ENABLE PARALLEL DML로 활성화해야 실제 병렬로 수행되며, 병렬 DML로 변경한 테이블은 같은 트랜잭션 안에서 다시 조회·변경할 수 없어 커밋 뒤에 접근해야 한다.

학교급식 제공 관리의 급식제공내역은 제공일자(DATE)로 분기 RANGE 파티셔닝하고, 그 아래를 학교코드로 HASH 서브파티셔닝(SUBPARTITIONS 8) 한 복합 파티션 테이블이다(아래 DDL). 각 선택지는 SELECT SUM(제공식수) FROM 급식제공내역 WHERE ... 형태의 조회가 어떤 파티션·서브파티션을 접근하는지를 설명한다. Partition Pruning이 설명대로 작동한다고 볼 때 가장 적절한 것은?

아래

```
CREATE TABLE 급식제공내역 (
  제공번호 NUMBER,
  제공일자 DATE,
  학교코드 NUMBER,
  식단구분 VARCHAR2(10),
  제공식수 NUMBER
)
PARTITION BY RANGE (제공일자)
SUBPARTITION BY HASH (학교코드) SUBPARTITIONS 8
(
  PARTITION p_2026q1 VALUES LESS THAN (DATE '2026-04-01'),
  PARTITION p_2026q2 VALUES LESS THAN (DATE '2026-07-01'),
  PARTITION p_2026q3 VALUES LESS THAN (DATE '2026-10-01'),
  PARTITION p_max VALUES LESS THAN (MAXVALUE)
);
```

- ① WHERE 제공일자 >= DATE '2026-07-01' 은 RANGE 프루닝으로 p_2026q3·p_max만 접근하고, 학교코드 조건이 없어도 옵티마이저가 두 파티션 각각에서 해시 서브파티션 1개씩만 골라 읽는다.
- ② WHERE 학교코드 = 30012 AND 식단구분 = '중식' 은 학교코드 등치로 각 파티션에서 서브파티션 1개씩 접근한 뒤, 식단구분 조건까지 있으니 그 서브파티션 안에서 한 번 더 서브파티션 프루닝이 걸려 읽을 세그먼트가 절반으로 준다.
- ③ WHERE 제공일자 = DATE '2026-05-10' AND 학교코드 = 30012 는 RANGE로 p_2026q2 한 파티션, HASH로 그 안의 서브파티션 1개까지 좁혀 최종 단일 서브파티션만 읽는다.
- ④ WHERE TO_CHAR(학교코드) LIKE '300%' 는 학교코드 앞 3자리가 '300'인 학교를 겨냥하므로, 그 값들이 매핑된 해시 서브파티션들만 골라 읽는 서브파티션 프루닝이 작동한다.

재난지원금 지급 정산 야간 배치가 지급신청(약 3억 2천만 건)과 지원유형(약 700건)을 지원유형코드로 조인해 지원유형별 지급총액을 집계한다. 지급신청은 지원유형 기준 파티션이 아니며, 지원유형은 소형 코드성 테이블이다. 아래 병렬 실행계획에서 두 테이블이 병렬 서버(Slave) 사이에 어떤 분배 방식으로 조인되는지 직접 지목한 설명으로 가장 적절한 것은?

아래

Id	Operation	Name	Pstart	Pstop	TQ	IN-OUT
0	SELECT STATEMENT					
1	PX COORDINATOR					
2	PX SEND QC (RANDOM)	:TQ10001			Q1,01	P->S
3	HASH GROUP BY				Q1,01	PCWP
* 4	HASH JOIN				Q1,01	PCWP
5	PX RECEIVE				Q1,01	PCWP
6	PX SEND BROADCAST	:TQ10000			Q1,00	P->P
7	PX BLOCK ITERATOR				Q1,00	PCWC
8	TABLE ACCESS FULL	지원유형			Q1,00	PCWP
9	PX BLOCK ITERATOR				Q1,01	PCWC
10	TABLE ACCESS FULL	지급신청			Q1,01	PCWP

- ① 소형 지원유형(Id 8)을 PX SEND BROADCAST(Id 6)로 모든 병렬 서버에 복제하고, 대형 지급신청(Id 10)은 재분배 없이 각 서버가 PX BLOCK ITERATOR(Id 9)로 자기 블록 범위만 읽어 조인하는 broadcast 분배다. 대형을 재분배하는 대신 700건짜리 소형을 복제하는 편이 싸기 때문이다.
- ② 조인 키 지원유형코드의 해시로 지급신청과 지원유형을 양쪽 모두 재분배(hash-hash) 한 뒤 각 서버가 자기 버킷 범위의 행만 Id 4 HASH JOIN에서 맞대며, 그 재분배가 Id 6의 :TQ10000과 Id 9에서 각각 일어나 두 갈래 테이블 큐로 나뉘고 Id 5 PX RECEIVE가 양쪽 결과를 함께 받는다.
- ③ 두 테이블이 지원유형코드로 동일하게 파티셔닝돼 있어 재분배 없이 대응 파티션 집합만 로컬로 조인하는 (full) 파티션 와이즈 조인이며, Id 8·Id 10이 같은 PX PARTITION 아래에서 읽힌다. Pstart·Pstop이 비어 있는 것도 각 서버가 파티션 쌍을 통째로 맡았다는 표시다.
- ④ 대형 지급신청만 Id 9 PX BLOCK ITERATOR로 각 서버가 파티션 단위로 읽고, 소형 지원유형을 지급신청의 파티션 경계에 맞춰 Id 6에서 PX SEND PARTITION (KEY)로 재분배하는 부분(partial) 파티션 와이즈 조인이다. Id 5 PX RECEIVE도 자기 파티션 몫만 받으므로 700건이 전 서버에 복제되지는 않는다.

분석함수(윈도우 함수)가 유발하는 WINDOW SORT와, 인덱스를 이용한 그 정렬의 대체에 대한 설명으로 가장 적절하지 않은 것은?

- ① 분석함수는 PARTITION BY·ORDER BY로 지정한 순서로 행을 늘어놓아야 윈도우를 적용할 수 있어, 실행계획에 WINDOW SORT 같은 정렬 오퍼레이션이 나타나는 것이 일반적이다.
- ② 분석함수의 PARTITION BY 컬럼과 ORDER BY 컬럼의 순서에 맞는 인덱스가 있어 그 정렬된 순서로 행을 공급받으면, 정렬 없이 처리하는 WINDOW NOSORT 형태로 바뀌어 정렬 단계가 생략될 수 있다.
- ③ 한 쿼리에 그룹 집계와 분석함수가 함께 있으면 SORT GROUP BY로 그룹 키 정렬이 이미 이뤄졌더라도, 분석함수가 요구하는 정렬 순서가 그와 다르면 별도의 WINDOW SORT가 추가로 필요할 수 있다.
- ④ 분석함수의 정렬은 관련 컬럼에 맞는 인덱스로 대체되므로, 서로 다른 PARTITION BY·ORDER BY를 쓰는 분석함수가 여러 개 든 쿼리라도 그 컬럼들에 인덱스만 갖춰 두면 실행계획에서 WINDOW SORT 없이 처리된다.

19

문화누리 이용권 관리 시스템의 이용권계정에는 이용자 가·나·다의 잔여지원액이 각각 100,000·200,000·300,000원(합 600,000)으로 들어 있다. 정산 세션 TX2가 t2에 SELECT SUM(지원액) FROM 이용권계정 FOR UPDATE(전 계정을 잠그고 합계를 읽는 정산 준비)를 발행하는데, 이미 t1에 다른 세션 TX1이 나·다의 지원액을 갱신해 두고 아직 커밋하지 않은 상태라 TX2는 대기한다. TX1이 t3에 커밋하면 TX2는 t4에 재개된다. t4에 TX2의 SELECT ... FOR UPDATE가 반환하는 SUM(지원액)으로 적절한 것은? (단, 오라클이며 격리수준은 기본값 READ COMMITTED이고 관련 Undo는 덮어써지지 않았다.)

아래

시각	TX1 (갱신 · 커밋)	TX2 (정산 SELECT ... FOR UPDATE)
t1	UPDATE 이용권계정 SET 지원액=지원액+50000 WHERE 이용자='나'; (나 200000→250000, 미커밋) UPDATE 이용권계정 SET 지원액=지원액+80000 WHERE 이용자='다'; (다 300000→380000, 미커밋)	
t2		SELECT SUM(지원액) FROM 이용권계정 FOR UPDATE; → 가는 잠그나 나·다는 TX1이 잠금 → 대기
t3	COMMIT; (나=250000, 다=380000 확정)	
t4		→ 잠금 획득 · SUM 반환
초기: 가=100000, 나=200000, 다=300000 (합 600000)		

- ① 600,000
- ② 650,000
- ③ 680,000
- ④ 730,000

20

상하수도 관망 정비 관리 시스템(SQL Server)에서 세션 53이 관로점검이력 테이블의 특정 관로 구간 행을 오래 갱신하는 트랜잭션을 열어 둔 사이, 두 세션(61·74)이 같은 자원을 요청하며 대기가 이어졌다. RCSI를 쓰지 않는 기본 READ COMMITTED 환경에서 sys.dm_exec_requests를 조회한 결과가 아래와 같다. 이 블로킹 상태에 대한 해석으로 가장 적절한 것은?

아래

[sys.dm_exec_requests - 관로점검이력 블로킹 체인]

session_id	blocking_session_id	wait_type	wait_time(ms)	status
53	0	(none)	0	running
61	53	LCK_M_X	18000	suspended
74	61	LCK_M_S	12000	suspended

- * blocking_session_id = 나를 막고 있는 세션 (0 = 막힌 것 없음)
- * LCK_M_X = 배타 잠금 대기, LCK_M_S = 공유 잠금 대기

- ① 세 세션이 모두 잠금(LCK_M_*) 대기 상태이므로 서로가 서로를 기다리는 교착(deadlock)이며, SQL Server 교착 모니터가 곧 한 세션을 희생자로 골라 롤백한다.
- ② blocking_session_id가 0이고 스스로는 대기하지 않는 53이 이 블로킹 체인의 선두이며, 61은 53에·74는 61에 막혀 있으므로, 53의 트랜잭션이 커밋·롤백으로 잠금을 풀면 61 → 74 순으로 대기가 풀려 나간다.
- ③ 74가 요청한 S 잠금은 공유 잠금이라 다른 잠금과 폭넓게 호환되므로, 74의 대기는 61이 아니라 순전히 53의 X 잠금 때문이며 61이 커밋해도 74는 계속 막힌다.
- ④ blocking_session_id가 0인 53도 실은 74에 간접적으로 막혀 세 세션이 순환 대기를 이루므로, wait_time이 가장 큰 53을 kill해야 전체 대기가 풀린다.

정답 및 해설

■ 정답 모아보기

1. ④	2. ③	3. ④	4. ②	5. ②
6. ①	7. ③	8. ①	9. ③	10. ①
11. ①	12. ②	13. ④	14. ②	15. ③
16. ③	17. ①	18. ④	19. ④	20. ②

문제 1. 정답 ④

두 기법이 무엇을 줄여 주고 무엇을 못 줄이는지를 갈라 봐야 합니다.

읽어야 할 데이터가 정해지면

방문할 블록 수(논리적 I/O) : 액세스 경로·스캔 범위가 결정 → 두 기법이 바꾸지 못함
 Multiblock I/O : 같은 블록들을 한 Call에 묶음 → I/O Call 횟수 절감
 Direct Path Read : 같은 블록들을 캐시 오버헤드 없이 PGA로 → 경유 비용 절감
 → 둘 다 '어떻게 읽는가(비용)'를 낮출 뿐, '몇 블록을 읽는가(양)'는 그대로

Multiblock I/O는 인접 블록을 한 I/O Call에 묶어 Call 횟수를 줄이고, Direct Path Read는 버퍼 캐시를 거치지 않아 경유 오버헤드를 덜 것입니다. 두 기법 모두 이미 읽기로 정해진 블록들을 더 싸게 읽어 줄 뿐, 방문해야 할 블록 수(논리적 I/O) 자체는 그대로입니다. 방문 블록 수를 실제로 줄이는 것은 ③이 말한 대로 스캔 범위·액세스 경로를 다듬는 일입니다.

④는 이 둘을 정반대로 뒤집어 "두 기법이 논리적 I/O 자체를 줄인다"고 했습니다. 이는 '읽는 양'과 '읽는 비용'이라는 다른 축을 하나로 뭉겐 서술이며, ③이 옳게 짚은 원리와 정면으로 어긋납니다.

①·②·③은 옳습니다. Multiblock I/O의 Call 절감(①), Direct Path Read의 캐시 우회와 그에 앞선 세그먼트 체크포인트(②), 그리고 I/O 효율화의 뿌리가 읽을 블록 수 자체를 줄이는 데 있다는 점(③)이 모두 블록 I/O 효율화의 실제 원리에 부합합니다.

선택지	판정	이유
①	○	Multiblock I/O는 인접 블록을 한 I/O Call에 묶어 읽어, 같은 블록 수를 읽어도 건별로 집어 오는 방식보다 Call 횟수를 줄입니다
②	○	Direct Path Read는 블록을 캐시에 올리지 않고 PGA로 곧바로 가져오되, 캐시에 남은 더티 블록을 먼저 내려쓰는 세그먼트 체크포인트가 앞서야 합니다
③	○	읽을 블록 수 자체는 스캔 범위·액세스 경로로 정해지며, 두 기법은 이미 읽어야 할 블록을 더 싸게 읽어 줄 뿐입니다
④	×	두 기법이 줄여 주는 것은 I/O Call 횟수·캐시 경유 비용이지 방문 블록 수(논리적 I/O)가 아닙니다. '읽는 비용'과 '읽는 양'을 뒤섞어 논리적 I/O가 준다고 한 정반대 서술입니다

■ 이 문제의 핵심

1. Multiblock I/O는 인접 블록을 한 Call에 묶어 I/O Call 횟수를 줄입니다.
2. Direct Path Read는 버퍼 캐시를 우회해 PGA로 곧바로 적재하되, 캐시의 더티 블록을 내려쓰는 세그먼트 체크포인트가 선행됩니다.
3. 두 기법은 '읽는 비용'을 낮출 뿐, 방문해야 할 블록 수(논리적 I/O) 자체는 바꾸지 못합니다.
4. 읽을 블록 수를 실제로 줄이는 것은 스캔 범위·액세스 경로 최적화입니다.

■ 한 줄 정리 Multiblock I/O와 Direct Path Read는 이미 읽어야 할 블록을 더 싸게 읽어 I/O Call·캐시 경유 비용을 줄일 뿐인데, "논리적 I/O(방문 블록 수) 자체를 줄인다"는 ④는 '읽는 비용'과 '읽는 양'을 뒤섞은 정반대 서술입니다.

문제 2. 정답 ③

각 프로세스가 실제로 손대는 대상을 이름의 인상과 분리해 봐야 합니다.

서버 프로세스 : SQL 파싱·실행, 블록을 DB 버퍼 캐시로 적재 (더티 버퍼 기록은 안 함)
 LGWR : Redo 로그 버퍼 → Redo 로그 파일 기록 (커밋 완료의 근거)
 DBWR : 더티 버퍼 → 데이터파일 기록 (커밋과 비동기, 느긋하게)
 CKPT : 체크포인트 시점에 데이터파일 헤더·컨트롤 파일의 SCN 갱신 (블록 실제 쓰기는 DBWR)

③은 LGWR의 역할과 커밋의 의미를 정확히 짚었습니다. 커밋이 보장하는 것은 변경 블록의 데이터파일 반영이 아니라 LGWR의 Redo flush입니다. 그래서 커밋은 로그만 내려쓰면 되어 빠르고(Fast Commit), 변경 블록은 DBWR가 나중에

비동기로 데이터파일에 반영해도 복구가 됩니다. 커밋의 완료 조건(Redo 기록)과 블록 기록(DBWR)이 분리돼 있다는 서술이 이 구조의 핵심입니다.

①·②·④는 각 이름이 불러일으키는 반사적 연상을 그대로 사실로 옮긴 함정입니다. "서버 프로세스가 자기 변경을 끝까지 책임진다"(①), "체크포인트라는 이름이니 블록을 직접 쓴다"(②), "블록을 쓰는 DBWR가 로그도 함께 남긴다"(④) — 모두 직관에서 곧장 튀어나오지만 실제 역할 분담과 어긋납니다.

선택지	판정	이유
①	×	서버 프로세스는 블록을 버퍼 캐시로 올려 처리할 뿐, 더티 버퍼를 데이터파일에 내려쓰는 일은 DBWR의 몫입니다. "서버 프로세스가 직접 기록한다"는 반사적 연상입니다
②	×	CKPT는 체크포인트 시 데이터파일 헤더·컨트롤 파일의 SCN을 갱신할 뿐, 더티 버퍼를 데이터파일에 실제로 써 내리는 것은 DBWR입니다. 이름에서 곧장 나온 오귀속입니다
③	○	LGWR가 커밋·로그버퍼 임계 시 Redo를 로그 파일에 기록하고, 커밋 완료는 이 Redo 기록으로 보장되며 블록의 데이터파일 반영과는 분리됩니다
④	×	Redo를 로그 파일에 쓰는 것은 LGWR이고, WAL은 '로그를 데이터보다 먼저'라는 순서 규칙일 뿐입니다. DBWR가 Redo까지 직접 쓴다는 것은 두 프로세스 역할을 뒤섞은 서술입니다

■ 이 문제의 핵심

1. 서버 프로세스는 SQL을 실행하고 블록을 DB 버퍼 캐시로 올릴 뿐, 더티 버퍼 기록은 하지 않습니다.
2. LGWR는 Redo 로그 버퍼를 로그 파일에 기록하며, 커밋 완료는 이 Redo 기록으로 보장됩니다.
3. DBWR가 더티 버퍼를 데이터파일에 쓰고, 이는 커밋과 비동기입니다.
4. CKPT는 체크포인트 시 SCN 갱신을 맡을 뿐 블록 실제 쓰기는 DBWR가 합니다.

■ 한 줄 정리 커밋 완료는 LGWR의 Redo flush로 보장되고 블록 기록(DBWR)과는 분리된다는 ③이 정확하며, "서버 프로세스가 직접 블록을 쓴다·CKPT가 블록을 쓴다·DBWR가 Redo도 쓴다"는 모두 이름에서 곧장 나온 반사적 연상입니다.

문제 3. 정답 ④

SQL 텍스트가 실행마다 다르므로(리터럴), 커서는 라이브러리 캐시에서 공유되지 못합니다. 그러면 파싱은 대부분 하드파싱이 됩니다. 먼저 하드파싱율을 셉니다.

하드파싱율 = 라이브러리 캐시 미스(비재귀) ÷ 비재귀 Parse 콜
= 499,000 ÷ 500,000 = 99.8%

→ 거의 매 실행이 계획을 새로 생성하는 하드파싱

소프트파싱만 반복되는 경우와는 정반대입니다. 여기서는 하드파싱이 데이터 디크너리(권한·통계·객체 정보)를 조회하는 재귀 statement를 대량으로 유발합니다. 재귀 측을 봅니다.

재귀 Call 비율 = 재귀 Total 콜 ÷ 비재귀 Total 콜 = 7,000,000 ÷ 1,000,000 = 7배

재귀 비중 = 7,000,000 ÷ (7,000,000 + 1,000,000) = 87.5%

→ 하드파싱이 디크너리 재귀 SQL 700만 콜을 낳았다(하드파싱의 부산물)

시간의 정체도 파싱 측입니다. 비재귀 Parse에 CPU 40.0초가 잡혀 있습니다.

Parse CPU 비중(비재귀) = 40.0 ÷ 49.0 × 100 ≈ 81.6% ← 계획 생성(하드파싱) CPU

Execute 비중 = 6.0 ÷ 49.0 × 100 ≈ 12.2% ← 실제 갱신은 소량

여기서 두 측을 분리해야 합니다.

측 A: 하드파싱(계획 생성·라이브러리 캐시 MISS·재귀 디크너리 콜) — 미스 99.8%, 재귀 7배 → 병목

측 B: 파싱 콜 횟수(소프트파싱·커서 미보유) — Parse:Execute=1:1이지만 여기선 소프트파싱이 아님

리터럴로 텍스트가 매번 달라 커서가 공유되지 않으니, Parse 콜을 캐싱으로 붙잡아도(측 B 처방)

텍스트가 다른 커서는 재사용되지 않는다. 원인은 측 A(하드파싱)이므로 처방도 측 A라야 한다.

처방은 커서 홀드가 아니라 리터럴을 바인드 변수로 바꿔 SQL 텍스트를 하나로 고정하는 것입니다. 텍스트가 같아지면 커서가 공유돼 하드파싱이 소프트파싱으로 바뀌고, 디크너리 재귀 콜(700만)과 Parse CPU(40초)가 함께 사라집니다. ④가 하드파싱 측(계획 생성·재귀 콜)을 정확히 지목했습니다.

선택지	판정	이유
①	×	Parse 콜과 Execute 콜이 각각 500,000으로 1:1인 것도 Parse Elapsed 42.0초가 49.0초의 85.7%인 것도 사실이지만, 라이브러리 캐시 미스 499,000(99.8%)이 말하듯 이 파싱들은 소프트파싱이 아니라 하드파싱입니다. 리터럴로 텍스트가 매번 달라 session_cached_cursors ·커서 홀드가 불잡을 공유 커서 자체가 없으니, 소프트파싱 빈도 축을 하드파싱 축으로 오인한 별개 축 혼동입니다
②	×	재귀 Query가 8,000,000 + 12,000,000 = 20,000,000으로 비재귀의 20배인 것은 맞지만 Disk 0의 논리 읽기이고, 재귀 statement 자체가 하드파싱이 부른 디렉터리 조회입니다. 원인이 아니라 결과이므로 버퍼 캐시를 키워도 하드파싱이 계속되는 한 재귀 23.0초는 다시 발생합니다 — 병목의 뿌리(하드파싱)가 아니라 부산물(재귀 논리 읽기)을 겨냥한 헛짚음입니다
③	×	Execute의 Query 1,000,000·Current 600,000(합계 1,600,000, 건당 3.2블록)과 Elapsed 7.0초는 사실이나 전체 49초의 14.3%뿐입니다. 병목은 실행 축이 아니라 Parse CPU 40초(81.6%)의 하드파싱 축이고, 재귀 Call 700만도 실행이 아니라 하드파싱의 부산물이라 UPDATE 실행계획을 다시 짜도 줄지 않습니다
④	○	하드파싱율 499,000/500,000 = 99.8%로 계획 생성 축이 병목임을 짚고, 그 하드파싱이 재귀 디렉터리 콜 700만(비재귀의 7배)을 유발했음을 연결한 뒤, 리터럴을 바인드 변수로 바꿔 커서를 공유시키는 처방이 정확합니다

■ 이 문제의 핵심

1. 하드파싱율 = 라이브러리 캐시 미스 ÷ Parse 콜. 여기서는 499,000/500,000 = 99.8%로, 리터럴 때문에 텍스트가 매번 달라 커서가 공유되지 못해 사실상 매 실행이 하드파싱입니다.
2. 하드파싱은 재귀 Call을 낳습니다. 계획을 새로 만들 때마다 데이터 디렉터리를 조회하므로 재귀 statement가 700만 콜(비재귀의 7배)로 폭증했습니다 — 재귀 축은 하드파싱의 부산물입니다.
3. 하드파싱 축(계획 생성)과 소프트파싱 빈도 축(커서 미보유)은 별개입니다. Parse:Execute = 1:1이어도 미스율이 99.8%면 커서 캐싱은 불잡을 공유 커서가 없어 무력합니다.
4. 처방은 커서 홀드·버퍼 캐시·실행계획 튜닝이 아니라 리터럴 → 바인드 변수입니다. 텍스트가 하나로 고정돼야 커서가 공유돼 하드파싱이 소프트파싱으로 바뀝니다.
5. Parse CPU 40초(81.6%)가 시간을 지배하고 Execute는 12.2%뿐이라, 병목은 실행이 아니라 파싱(계획 생성)에 있습니다.

- 한 줄 정리 하드파싱율이 99.8%(499,000/500,000)이고 재귀 디렉터리 콜이 700만(비재귀의 7배)으로 폭증한 것은 리터럴 때문에 커서가 공유되지 못해 매 실행이 하드파싱이기 때문이며, 처방은 커서 홀드·버퍼 캐시가 아니라 리터럴을 바인드 변수로 바꿔 하드파싱을 소프트파싱으로 돌리는 것이다.

문제 4. 정답 ②

'예상'과 '실측'을 가르는 기준은 문장을 실제로 수행했는가입니다.

예상 계획 : EXPLAIN PLAN → PLAN_TABLE → DBMS_XPLAN.DISPLAY (미수행, 추정 행 수만)

실제 계획 : gather_plan_statistics + 실제 수행

→ DBMS_XPLAN.DISPLAY_CURSOR(format=>'ALLSTATS LAST')

→ 단계별 예상 행 수 · 실측 행 수 동시 표시 → 추정 오차 진단

V\$SQL_PLAN : 파싱·수행으로 라이브러리 캐시에 올라온 '실제 커서'의 계획 (PLAN_TABLE과 별개)

SQL 모니터 : 실행 중·장시간 문장의 실시간 실측(단계별 행 수·시간) 리포트

②는 이 경계를 정확히 지킵니다. **gather_plan_statistics** 힌트(또는 **STATISTICS_LEVEL=ALL**)로 실행 통계 수집을 켜고 문장을 실제로 수행한 뒤, **DISPLAY_CURSOR**를 'ALLSTATS LAST' 포맷으로 조회하면 옵티마이저의 예상 행 수와 실제로 흘러온 실측 행 수가 단계별로 나란히 찍힙니다. 이 둘의 차이가 벌어진 단계가 곧 추정이 빛나간 지점이고, 튜닝의 출발점이 됩니다.

①·③·④는 예상과 실제의 경계를 뭉갠습니다. 실측 행 수는 힌트만으로 얻어지지 않고 반드시 실제 수행이 있어야 하며(①), **V\$SQL_PLAN**은 PLAN_TABLE의 예상 계획이 아니라 수행으로 캐시에 올라온 실제 커서의 계획이고(③), 실시간 SQL 모니터링은 미수행 추정 리포트가 아니라 실행 중 문장의 실측을 담는 리포트입니다(④).

선택지	판정	이유
①	×	gather_plan_statistics 가 남기는 실측 행 수는 문장을 수행해야 채워집니다. 미수행 상태로 실측을 비교할 수 있다는 것은 예상·실측의 경계를 뭉개 서술입니다
②	○	힌트로 통계 수집을 켜고 실제 수행한 뒤 DISPLAY_CURSOR('ALLSTATS LAST') 로 조회하면 단계별 예상 행 수와 실측 행 수를 함께 보아 추정 오차를 짚을 수 있습니다
③	×	V\$SQL_PLAN 은 파싱·수행으로 라이브러리 캐시에 올라온 실제 커서의 계획이며, EXPLAIN PLAN 이 PLAN_TABLE 에 남기는 예상 계획과 출처가 다릅니다
④	×	실시간 SQL 모니터링은 실행 중·장시간 문장의 단계별 실측 행 수·소요 시간을 담은 실측 리포트이지 미수행 추정 리포트가 아닙니다

■ 이 문제의 핵심

1. 예상 계획은 **EXPLAIN PLAN**→**DISPLAY**(미수행, 추정 행 수), 실제 계획은 실제 수행 뒤 **DISPLAY_CURSOR**로 봅니다.
2. **gather_plan_statistics**(또는 **STATISTICS_LEVEL=ALL**) + **DISPLAY_CURSOR('ALLSTATS LAST')**로 단계별 예상·실측 행 수를 나란히 비교합니다.
3. 실측 행 수는 실제 수행이 있어야 채워집니다 — 힌트만으로는 나오지 않습니다.
4. **V\$SQL_PLAN**은 캐시의 실제 커서 계획, SQL 모니터링은 실행 중 문장의 실측 리포트입니다.

■ 한 줄 정리 **gather_plan_statistics**로 실행 통계를 켜고 실제 수행한 뒤 **DISPLAY_CURSOR('ALLSTATS LAST')**로 예상·실측 행 수를 함께 보는 ②가 정확하며, 실측을 미수행 상태에서 얻는다거나 **V\$SQL_PLAN**·SQL 모니터를 예상 계획으로 본 나머지는 예상·실제의 경계를 뭉개 서술입니다.

문제 5. 정답 ②

먼저 BCHR로 논리 대비 물리 읽기를 봅니다.

$$\begin{aligned} \text{BCHR} &= ((\text{Query} + \text{Current}) - \text{Disk}) / (\text{Query} + \text{Current}) \times 100 \\ &= (1,000,000 + 0 - 700,000) / 1,000,000 \times 100 \\ &= 300,000 / 1,000,000 \times 100 = 30.0\% \end{aligned}$$

$$\text{물리 읽기 비중} = 700,000 / 1,000,000 \times 100 = 70.0\%$$

캐시 적중이 30%로 낮습니다. 논리 읽기 100만 중 70만이 실제 디스크에서 올라온 물리 읽기라는 뜻입니다. 다음은 시간의 정체입니다.

$$\begin{aligned} \text{CPU 비중} &= 10.000 / 60.000 \times 100 \approx 16.7\% \quad \rightarrow \text{나머지 83\%가 대기} \\ \text{Elapsed} - \text{CPU} &= 60.000 - 10.000 = 50.0\text{초} \quad \leftarrow \text{기다린 시간} \end{aligned}$$

응답의 83%가 대기입니다. 대기가 난 곳을 Row Source로 찾되, pr(물리)과 pw(temp)를 분리해서 봅니다.

pw(temp spill) = 모든 노드 0 → spill 없음

pr(물리 읽기) = 등급판정이력 Full Scan 700,000 (도착장 0, 조인·집계 증가분 0)

→ 물리 읽기 700,000 전부가 큰 테이블 Full Scan의 디스크 읽기

등급판정이력 Full Scan time = 50.0초 (전체 60초의 약 83%)

pw=0이 모든 노드에 붙어 있어 spill은 없습니다. 물리 읽기 700,000은 조인·집계가 아니라 캐시에 없는 **등급판정이력**을 디스크에서 읽어 올린 것이고, 그 노드에서 50초가 났습니다. cr은 이 Full Scan(999,000)이 지배하지만, 논리 읽기가 곧 병목은 아닙니다 — BCHR 30%가 말하듯 시간을 만든 것은 물리 디스크 I/O입니다.

논리 읽기(cr) 지배 = 등급판정이력 999,000 (99.9%) ← 크지만 대부분 물리로 실현됨

물리 읽기(pr) 정체 = 등급판정이력 700,000 → time 50초

따라서 처방은 물리 읽기 자체를 줄이는 것 — 스캔 대상을 좁히는 파티션 프루닝이나 캐시 적중 개선입니다. ②가 BCHR과 병목 지목을 함께 옳게 했습니다.

선택지	판정	이유
①	×	Fetch 2,000회에 CPU 10초가 잡힌 것은 맞지만 그 10초는 Elapsed 60초의 16.7%뿐이고, 나머지 83%(50초)는 등급판정이력 Full Scan의 물리 읽기 대기입니다. 해시 매칭·집계가 CPU를 쓴다는 사실을 CPU 바운드 진단으로 비약해 DOP를 처방한 것으로, 대기의 원인을 CPU로 오귀속했습니다(병렬 스캔이 물리 I/O를 분산할 수는 있어도, 그 근거가 CPU라는 진단이 틀렸습니다)
②	○	BCHR (1,000,000-700,000)/1,000,000 = 30.0%로 캐시 적중이 낮고, pw=0으로 spill이 없으며 pr 700,000이 모두 등급판정이력 Full Scan(time 50초)에 붙어 있음을 짚어 물리 I/O를 병목으로 지목하고 물리 읽기 축소를 처방한 것이 정확합니다
③	×	cr 999,000(99.9%)이 등급판정이력 Full Scan이고 도착장 은 1,000, 그 노드 time이 50초인 것까지 사실이나, BCHR 30%가 보여 주듯 그 논리 읽기의 70%(700,000)는 이미 물리 디스크 읽기로 실현됐고 CPU는 16.7%뿐이라 나머지 50초는 그 디스크 대기입니다. 물리 읽기를 논리 읽기에 딸린 부차적 수치로 미룬 것은 시간을 만든 축(물리 I/O)을 논리 읽기로 오귀속한 해석입니다
④	×	HASH GROUP BY 의 cr 1,000,000·time 59.5초는 맞지만 그 노드는 pr=0·pw=0이라 temp spill 자체가 없습니다. 물리 읽기 700,000은 집계가 아니라 등급판정이력 Full Scan(pr=700,000)에서 났으므로, PGA를 키워도 60초는 그대로입니다 — 물리 스캔 I/O를 집계 spill로 오귀속한 것입니다

■ 이 문제의 핵심

- BCHR = ((Query+Current)-Disk)/(Query+Current) × 100. 여기서는 (1,000,000-700,000)/1,000,000 = 30.0%로 캐시 적중이 낮아, 논리 읽기의 70%가 실제 디스크 읽기입니다.
- cr(논리)과 pr(물리)·pw(temp)를 분리해야 합니다. cr은 **등급판정이력** Full Scan(99.9%)이 지배해도, 시간을 만든 건 그 노드의 pr=700,000 물리 읽기입니다.
- pw**가 어느 노드에 붙었는지가 핵심입니다. 모든 노드가 pw=0이라 spill은 없고, 병목은 집계가 아니라 캐시에 없는 큰 테이블의 물리 스캔입니다.
- BCHR이 낮다 = 물리 I/O 바운드입니다 — 논리 읽기가 커도 그게 캐시에서 나오면 빠르지만, 여기서 70%가 디스크에서 올라와 50초가 걸렸습니다.
- 처방은 물리 읽기 축소(파티션 프루닝·캐시 적중 개선)입니다. DOP·논리 읽기·PGA는 50초의 물리 스캔 대기를 근거부터 잘못 겨냥합니다.

- 한 줄 정리 BCHR 30%로 캐시 적중이 낮고 pw=0이라 spill이 없으며 물리 읽기 700,000이 모두 **등급판정이력** Full Scan(time 50초)에 붙어 있으므로 병목은 물리 디스크 I/O이고, 이를 CPU·논리 읽기·집계 spill로 돌리면 원인을 오귀속하게 된다.

문제 6. 정답 ①

플랜의 두 노드에서 나온 Card 값을 나란히 놓습니다.

INDEX RANGE SCAN 신청_IX Card 9,500 ← 인덱스에서 훑고 남은 건수
TABLE ACCESS BY INDEX ROWID 발급신청 Card 2,800 ← 테이블까지 거친 뒤 남은 건수

신청_IX는 (발급기관코드, 접수일시) 순서입니다. WHERE에는 **발급기관코드 = 'A21'**(등치)와 **접수일시** 범위가 있으므로, 선두 칼럼 등치 + 두 번째 칼럼 범위까지가 인덱스로 연속해서 훑을 수 있는 구간입니다. 그 결과가 INDEX RANGE SCAN의 9,500건입니다.

긴급여부는 **신청_IX**의 구성 칼럼이 아닙니다. 인덱스에 값이 없으니 인덱스 단계에서는 걸러낼 수 없고, ROWID로 테이블 블록을 읽어 온 TABLE ACCESS BY INDEX ROWID 단계에서 필터로 처리됩니다. 그래서 9,500건이 이 단계를 거치며 2,800건으로 줄어듭니다.

9,500 (인덱스 액세스: 발급기관코드=, 접수일시 범위)
└─ 테이블 필터(긴급여부='Y') 적용 → 2,800 (최종)

즉 ①이 이 구조를 정확히 서술합니다. 긴급여부까지 인덱스 액세스 조건으로 끌어올린 ②, 인덱스 반환 건수를 최종 건수로 본 ③, 선두 칼럼 등치까지 필터로 밀어 넣은 ④는 이 Card 흐름과 어긋납니다.

선택지	판정	이유
①	○	발급기관코드(등치)·접수일시(범위)는 인덱스 액세스 조건, 긴급여부는 인덱스에 없어 TABLE ACCESS 단계 필터 — Card가 9,500→2,800으로 줄어드는 흐름과 일치합니다
②	×	긴급여부는 신청_IX 구성 칼럼이 아니므로 스캔 시작·종료점을 정하는 인덱스 액세스 조건이 될 수 없습니다. 테이블 필터인 긴급여부를 인덱스 액세스 조건 집합에 끼워 넣은 진술로, 그것까지 스캔 범위를 좁혔다면 Id 3 Card가 이미 2,800이었을 텐데 실제로는 9,500입니다
③	×	Card 9,500은 Id 3 인덱스 단계 값일 뿐이고, 긴급여부 필터를 거친 Id 2의 Card는 2,800으로 찍혀 있어 테이블 액세스 단계에서 건수 감소가 분명히 일어납니다. Cost 350과 26의 차이는 랜덤 액세스 비용일 뿐, 건수가 그대로라는 근거가 되지 못합니다
④	×	두 번째 칼럼이 범위라는 성질을 선두 칼럼에까지 확대한 진술입니다. 선두 칼럼 등치는 그 자체로 액세스 조건이고, 범위는 그 다음 칼럼부터 적용됩니다. 등치까지 필터였다면 인덱스를 전 구간 훑어야 해 Id 3이 Cost 26·Card 9,500으로 끝날 수 없고, INDEX RANGE SCAN도 성립하지 않습니다

■ 이 문제의 핵심

- 결합 인덱스는 선두 칼럼 등치 + 다음 칼럼 범위까지가 액세스 조건. 여기서는 발급기관코드(=)·접수일시(범위)가 INDEX RANGE SCAN 구간을 만듭니다(Card 9,500).
- 인덱스에 없는 칼럼은 테이블 필터. 긴급여부는 ROWID로 테이블을 읽은 뒤 TABLE ACCESS BY INDEX ROWID 단계에서 걸러집니다.
- 두 노드의 Card 차이(9,500 vs 2,800)가 곧 테이블 필터의 효과입니다. 인덱스 반환 건수 ≠ 최종 건수.
- 테이블 필터 조건을 인덱스 액세스 조건 집합에 끼워 넣지 않습니다. 인덱스에 없는 긴급여부는 스캔 범위를 정하는 데 관여하지 못합니다.

■ 한 줄 정리 선두 칼럼 등치(발급기관코드)와 다음 칼럼 범위(접수일시)는 인덱스 액세스 조건이 되고, 인덱스에 없는 긴급여부는 테이블 액세스 단계 필터가 되어 Card가 9,500에서 2,800으로 줄어듭니다.

문제 7. 정답 ③

인덱스는 잘 땀습니다. 그런데도 70초가 걸린 이유를 클러스터링 팩터와 손익분기점 두 지표로 봅니다. 인덱스 cr은 테이블 노드에 누적되므로 테이블 랜덤 블록만 떼어 냅니다.

테이블 랜덤 블록(cr) = TABLE 노드 cr - INDEX 노드 cr

$$= 1,803,000 - 3,000 = 1,800,000$$

 CF 밀도 = 테이블 랜덤 블록(cr) ÷ ROWID 수

$$= 1,800,000 \div 2,000,000 = 0.90 \quad \leftarrow \text{최댓값 1에 근접(최악)}$$

 완전 정렬 시 밀도 = 테이블 총 블록 ÷ 테이블 총 행

$$= 200,000 \div 20,000,000 = 0.01 \quad (\text{블록당 100행})$$

 약화 배수 = $0.90 \div 0.01 = 90\text{배}$

CF 밀도 0.90은 거의 1 — 인덱스로 뽑은 행 하나하나가 저마다 다른 테이블 블록에 흩어져 있어, 2,000,000행을 읽으며 1,800,000번의 테이블 랜덤 단일블록 액세스가 일어났습니다. 이제 손익분기점을 봅니다. 같은 데이터를 Full Scan으로 읽으면 몇 블록일까요.

선택도 = 반환 행 2,000,000 ÷ 테이블 총 행 20,000,000 = 10%
 인덱스 경우 테이블 블록 I/O = 1,800,000 (단일블록 랜덤)
 Full Scan 시 블록 I/O = 200,000 (멀티블록)
 손익분기 배수 = $1,800,000 \div 200,000 = 9.0\text{배} \rightarrow$ 손익분기점을 크게 초과

인덱스로 읽으면 흩어진 블록을 1,800,000번 단일블록으로 방문하지만, Full Scan은 테이블 전체를 200,000블록 멀티블록으로 훑습니다. 9배 더 적은 블록을, 그것도 멀티블록으로 읽으니 이 선택도(10%)에서는 인덱스가 이미 손익분기점을 넘어 손해입니다. 시간·논리 읽기가 모두 테이블 랜덤 액세스에 몰립니다.

INDEX RANGE SCAN cr = 3,000 (전체 cr 1,803,000의 0.17%) ← 인덱스는 가벼움
 TABLE 랜덤 액세스 cr = 1,800,000 (99.8%) ← 논리 읽기의 대부분
 TABLE 랜덤 액세스 time = 66초 (전체 70초의 약 94%) ← 시간의 대부분
 CPU 비중 = $18.0 / 70.0 \times 100 \approx 25.7\%$ → 대기 바운드

두 지표가 함께 말합니다: CF 밀도(0.90)가 최악이라 테이블 랜덤 액세스가 폭증했고, 10% 선택도에서 손익분기점을 9배 초과해 인덱스가 오히려 손해입니다. 처방은 인덱스 리빌드가 아니라, Full Scan 전환이거나 인덱스 키 순서로 테이블을 재정렬(CF 개선)하거나 조희 컬럼을 담은 커버링 인덱스로 테이블 액세스 자체를 없애는 것입니다. ③이 두 지표를 옮겨 결합했습니다.

선택지	판정	이유
①	×	pr이 테이블 250,000·인덱스 800으로 테이블 쪽에 몰린 것도 그 노드 time 66초도 사실이지만, 250,000을 캐시로 돌려도 CF 밀도 0.90이 만든 1,800,000번의 논리적 테이블 랜덤 블록 방문 자체는 남습니다. 병목의 원인(나쁜 CF·손익분기점 초과)을 물리 읽기로 오귀속한 처방입니다
②	×	cr 1,803,000이 크고 Fetch 콜당 약 900블록인 것은 맞지만 그 99.8%(1,800,000)는 인덱스가 아니라 테이블 랜덤 액세스입니다. INDEX 스캔 cr은 3,000(0.17%)뿐이라 리빌드로 리프를 다져 그 3,000을 0에 가깝게 만들어도 1,800,000번의 테이블 블록 방문은 그대로 남고, Disk 250,000도 그 랜덤 방문이 부른 물리 읽기라 원인을 인덱스로 오귀속한 것입니다
③	○	CF 밀도 $1,800,000/2,000,000 = 0.90$ (최댓값 1에 근접)로 테이블 랜덤 액세스를 지목하고, 10% 선택도에서 인덱스 경우 1,800,000블록이 Full Scan 200,000블록의 9배라 손익분기점을 넘었음을 짚어 Full Scan 전환·CF 개선·커버링을 처방한 것이 정확합니다
④	×	CPU 18초는 Elapsed 70초의 25.7%뿐이라 CPU 바운드가 아닙니다. 74%가 테이블 랜덤 액세스의 I/O 대기이고, 인덱스 노드 time 1.2초가 짧다는 사실은 인덱스가 멀쩡하다는 근거일 뿐 CPU 바운드의 근거가 아닙니다. DOP 상향은 대기의 원인(나쁜 CF)을 CPU로 오귀속한 것입니다

■ 이 문제의 핵심

1. CF 밀도 = 테이블 랜덤 블록(cr) ÷ ROWID 수. 테이블 cr은 인덱스 cr을 뺀 1,800,000이고, $1,800,000/2,000,000 = 0.90$ 으로 최댓값 1에 근접합니다(완전 정렬 시 $200,000/20,000,000 = 0.01$ 의 90배).
2. 인덱스 손익분기점 = 인덱스 경우 테이블 블록 vs Full Scan 블록. 인덱스로는 1,800,000블록(단일블록 랜덤), Full Scan은 200,000블록(멀티블록)이라 9배 차이 — 10% 선택도에서 인덱스가 이미 손해입니다.
3. CF 밀도가 1에 가까우면 인덱스로 뽑은 행마다 서로 다른 블록에 흩어져 테이블 랜덤 액세스가 폭증합니다 — 시간(66초)·논리 읽기(99.8%)가 모두 이 노드에 몰립니다.
4. 두 지표를 함께 봐야 진단이 유일해집니다. CF로 랜덤 액세스의 폭증을 확인하고, 손익분기점으로 인덱스보다 Full Scan이 유리함을 확인합니다.
5. 처방은 Full Scan 전환 또는 인덱스 키 순서 재정렬(CF 개선)·커버링 인덱스입니다. 버퍼 캐시·인덱스 재생성·DOP는 1,800,000번의 랜덤 블록 방문을 없애지 못합니다.

■ 한 줄 정리 CF 밀도 0.90($1,800,000/2,000,000$)으로 테이블 랜덤 액세스가 폭증하고, 10% 선택도에서 인덱스 경우 1,800,000블록이 Full Scan 200,000블록의 9배라 손익분기점을 넘었으므로 병목은 인덱스가 아니라 테이블 랜덤 액세스이며, 버퍼 캐시·인덱스 재생성·DOP는 병목을 잘못 짚은 처방이다.

문제 8. 정답 ①

커버링이 성능을 높이는 이유는 바로 테이블 되짚기(Random 액세스)를 없애기 때문입니다.

커버링 아님 : 인덱스에서 ROWID 획득 → ROWID로 테이블 블록 임의 방문(Random 액세스) → 컬럼 값 획득
커버링 : 필요한 컬럼이 인덱스에 모두 있음 → 리프만 읽고 종료
→ ROWID로 테이블 블록을 방문하는 단계가 사라짐 → Random 액세스 제거

인덱스 전용 스캔의 핵심은 조회에 필요한 컬럼이 인덱스 안에 다 들어 있어, 리프에서 얻은 ROWID로 테이블 블록을 다시 찾아가는 단계 자체를 없앤다는 데 있습니다. 그래서 흩어진 테이블 블록을 건별로 집어 오는 Random 액세스가 사라지고 조회 부담이 줄어듭니다.

- ①은 이 효과를 정반대로 뒤집었습니다. "커버링이면서도 ROWID로 테이블을 한 번씩 방문해 Random 액세스가 늘어난다"고 했지만, 테이블 방문을 없애는 것이 바로 커버링의 목적입니다. ROWID로 테이블을 되짚는 것은 커버링이 아닐 때의 동작이며, ①은 그 되짚기를 커버링에 붙여 이득을 손해로 둔갑시킨 서술입니다.
- ②·③·④는 옳습니다. 선두 컬럼 조건이 없어도 커버링이면 Index Full Scan으로 테이블 없이 처리 가능하고 작은 인덱스가 Full Table Scan보다 유리하다는 점(②), Index Fast Full Scan의 물리적·Multiblock·병렬 설정과 NOT NULL 컬럼 요건(③), 그리고 커버링 가능 여부가 스캔 방식과 별개라는 점(④)이 모두 인덱스 스캔의 실제 동작에 부합합니다.

선택지	판정	이유
①	×	커버링은 필요한 컬럼이 인덱스에 다 있어 ROWID로 테이블을 되짚는 단계를 없애 Random 액세스를 제거합니다. "테이블을 방문해 Random 액세스가 느다"는 것은 커버링의 이득을 손해로 뒤집은 정반대 서술입니다
②	○	선두 컬럼 조건이 없어도 커버링이면 Index Full Scan으로 리프만 훑어 처리할 수 있고, 인덱스가 테이블보다 작으면 Full Table Scan보다 읽을 블록이 적습니다
③	○	Index Fast Full Scan은 세그먼트 블록을 물리적 순서로 Multiblock·병렬로 읽되, NULL 행 누락을 막기 위해 인덱스 컬럼 중 하나 이상이 NOT NULL이어야 합니다
④	○	커버링 가능 여부는 스캔 방식과 별개라, 필요한 컬럼이 인덱스에 다 있으면 Range Scan이든 Full Scan이든 테이블 액세스를 생략할 수 있습니다

■ 이 문제의 핵심

1. 인덱스 전용 스캔(커버링)은 필요한 컬럼이 인덱스에 다 있어 ROWID 테이블 되짚기를 없애 Random 액세스를 제거합니다.
2. Index Full Scan은 선두 컬럼 조건이 없어도 커버링이면 리프를 연결 순서로 훑어 테이블 없이 처리할 수 있습니다.
3. Index Fast Full Scan은 세그먼트를 물리적 순서로 Multiblock·병렬로 읽되, NOT NULL 컬럼이 있어야 테이블을 대신합니다.
4. 커버링 가능 여부는 스캔 방식과 별개입니다.

■ 한 줄 정리 커버링의 본질은 ROWID 테이블 되짚기를 없애 Random 액세스를 제거하는 것인데, "커버링하면서 테이블을 방문해 Random 액세스가 늘어난다"는 ①은 그 이득을 정반대로 뒤집은 서술입니다.

문제 9. 정답 ③

제약을 뒷받침하는 인덱스가 제약 삭제 시 어떻게 되는지는 그 인덱스가 어떻게 생겼는가에 따라 갈립니다.

제약 생성 시 오라클이 함께 자동 생성한 인덱스

→ 제약을 DROP하면 그 인덱스도 함께 삭제됨 (KEEP INDEX를 명시해야 남김)

기존에 있던 인덱스를 제약이 재사용한 경우

→ 제약을 DROP해도 그 인덱스는 원래부터 독립 오브젝트라 그대로 남음

∴ '남는다'는 것은 후자의 사례일 뿐, 모든 경우의 규칙이 아님

인덱스가 제약 삭제 후에도 남는 것은 제약이 기존 인덱스를 재사용했을 때의 이야기입니다. 제약을 만들 때 오라클이 함께 자동 생성한 인덱스라면, 제약을 DROP할 때 그 인덱스도 같이 사라집니다(그대로 두려면 **DROP CONSTRAINT ... KEEP INDEX**를 명시해야 합니다).

③은 이 부분 사례를 전체 규칙으로 부풀렸습니다. "생성 경위와 상관없이 항상 그대로 남는다"는 서술은, 재사용된 인덱스만 성립하는 성질을 자동 생성된 인덱스까지 싸잡아 늘려 붙인 과잉 일반화입니다.

①·②·④는 옳습니다. 같은 구성의 유니크 인덱스가 있으면 재사용한다는 점(①), 지연 제약이나 제약 컬럼을 선두에 둔 비유니크 인덱스로도 제약을 지원할 수 있다는 점(②), 제약은 무결성 규칙이고 인덱스는 중복 검사 수단이라 삽입 시 인덱스로 중복을 확인한다는 점(④)이 모두 제약과 인덱스의 실제 관계에 부합합니다.

선택지	판정	이유
①	○	PK 제약 생성 시 인덱스가 자동으로 만들어지되, 같은 컬럼 구성의 유니크 인덱스가 이미 있으면 그것을 재사용합니다
②	○	지연(DEFERRABLE) 제약이거나 제약 컬럼을 선두에 포함한 비유니크 결합 인덱스가 있으면, 비유니크 인덱스로도 제약을 지원할 수 있습니다
③	×	자동 생성된 인덱스는 제약을 DROP하면 함께 삭제됩니다(KEEP INDEX 명시 시 유지). '항상 남는다'는 것은 재사용 인덱스만의 사례를 전체로 부풀린 과잉 일반화입니다
④	○	제약은 중복을 막는 규칙, 인덱스는 그 규칙을 빠르게 검사하는 수단이라, 삽입 시 제약 컬럼 값이 인덱스에 있는지 확인해 중복이면 거부합니다

■ 이 문제의 핵심

1. PK 제약을 만들면 지원 인덱스가 자동 생성되되, 같은 구성의 유니크 인덱스가 있으면 재사용합니다.
2. 제약은 유니크 인덱스로만 뒷받침되는 것이 아니라, 지연 제약·비유니크 결합 인덱스로도 지원됩니다.
3. 제약과 인덱스는 별개 오브젝트이나, 자동 생성된 인덱스는 제약 삭제 시 함께 사라집니다(KEEP INDEX로 유지).
4. 제약은 무결성 규칙, 인덱스는 그 규칙의 중복 검사 수단입니다.

- 한 줄 정리 제약 삭제 후 인덱스가 남는 것은 기존 인덱스를 재사용했을 때뿐이고 자동 생성된 인덱스는 함께 삭제되는데, "생성 경위와 상관없이 항상 남는다"는 ③은 부분 사례를 전체 규칙으로 부풀린 과잉 일반화입니다.

문제 10. 정답 ①

Id 1의 오퍼레이션 이름을 그대로 읽습니다.

Id 1 NESTED LOOPS ← 조인 방식: NL
 Id 2~3 검사대상자 (INDEX RANGE SCAN) 구동 집합: 각 행을 하나씩 뺌
 Id 4~5 판정결과 (INDEX UNIQUE SCAN 판정결과_PK) 구동: 행마다 1건씩 탐침

최상단 조인 노드가 **NESTED LOOPS**입니다. NL 조인은 구동 테이블(검사대상자)에서 한 행을 뺌 때마다 안쪽 테이블(판정결과)을 **판정결과_PK** 인덱스로 파고들어 매칭 행을 찾습니다. 그래서 안쪽 접근이 **INDEX UNIQUE SCAN**으로, 대상자ID 하나당 한 건씩 반복됩니다.

①은 이 플랜을 해시 조인으로 뒤바꿔 서술합니다. 해시 조인이라면 한쪽을 해시 테이블로 미리 적재하고 다른 쪽을 스캔하며 탐침하는 **HASH JOIN** 노드가 있어야 하고, 안쪽을 인덱스로 한 건씩 반복 탐색하지도 않습니다. 이 플랜엔 **HASH JOIN** 노드도 없고, 오히려 안쪽 판정결과를 인덱스로 반복 탐침하는 전형적 NL 구조입니다. 따라서 "가장 적절하지 않은 것"은 ①입니다.

- ① 해시 조인 + 반복 탐침 없음 : 노드가 HASH JOIN이어야 성립 → 이 플랜은 NESTED LOOPS (틀림)
- ② NESTED LOOPS, 구동 행마다 탐침 : Id 1·Id 5와 일치 ○
- ③ 안쪽 인덱스(판정결과_PK) 필요 : INDEX UNIQUE SCAN 근거 ○
- ④ 검사대상자→판정결과, 작은 구동 : NL 선택 맥락과 일치 ○

②③④는 NL 조인의 동작(구동 행마다 탐침, 안쪽 인덱스 필요, 작은 구동 집합에 유리)을 정확히 서술합니다. ①만 NL을 해시로 오독해 값을 뒤바꿨습니다.

선택지	판정	이유
①	×	Id 1은 NESTED LOOPS 이지 HASH JOIN 이 아닙니다. 해시 조인은 한쪽을 해시 테이블로 적재하지만, 이 플랜은 구동 행마다 판정결과를 판정결과_PK 로 반복 탐침합니다 — NL을 해시로 뒤바꾼 값스왑 진술입니다
②	○	Id 1 NESTED LOOPS 가 맞고, 구동 검사대상자(Id 2~3)의 각 행마다 판정결과(Id 4~5)를 한 번씩 탐침하는 NL 동작과 일치합니다
③	○	안쪽 판정결과 접근이 INDEX UNIQUE SCAN 판정결과_PK 이므로, 조인 칼럼(대상자ID) 인덱스가 있어야 이 반복 탐침 플랜이 성립합니다
④	○	Id 2가 위, Id 4가 아래인 조인 순서(검사대상자→판정결과)와, 구동 집합 3만 건 수준에서 반복 탐침이 감당되는 NL 선택 맥락이 맞습니다

▪ 이 문제의 핵심

1. 조인 방식은 최상단 조인 노드 이름으로 판별합니다. Id 1이 **NESTED LOOPS**면 NL, **HASH JOIN**이면 해시 — 이 플랜은 NL입니다.
2. NL 조인은 구동 행마다 안쪽을 반복 탐침합니다. 검사대상자 한 행을 읽을 때마다 판정결과를 **판정결과_PK**로 한 건씩 찾습니다.
3. 안쪽 조인 칼럼 인덱스가 NL의 전제입니다. **INDEX UNIQUE SCAN 판정결과_PK**가 없으면 대상자ID당 1건 탐침이 성립하지 않습니다.
4. NL ≠ 해시 조인. 해시 조인은 한쪽을 해시 테이블로 적재해 반복 탐침을 없애는 방식으로, **HASH JOIN** 노드가 있어야 합니다 — 이 플랜엔 없습니다.

▪ 한 줄 정리 Id 1이 **NESTED LOOPS**이고 안쪽 판정결과를 **판정결과_PK**로 구동 행마다 반복 탐침하는 NL 조인이므로, 이를 해시 조인(한쪽 해시 적재·반복 탐침 없음)으로 서술한 ①이 가장 적절하지 않다.

문제 11. 정답 ①

해시 조인의 처리 골격은 Build → Probe 두 단계이고, 각 입력을 훑는 횟수가 이 방식의 성격을 결정합니다.

Build 단계 : Build Input 1회 스캔 → Hash Area에 해시 테이블 구축
 Probe 단계 : Probe Input 1회 스캔 → 각 행의 키로 해시 테이블을 조회해 매칭
 ⇒ 두 입력을 각각 한 번씩만 읽음 (NL처럼 상대를 되읽는 반복 스캔이 없음)

①은 이 입력별 1회 읽기 성질을 정확히 짚습니다. NL 조인은 드라이빙 행마다 상대 집합을 되읽어 접근 횟수가 드라이빙 건수에 비례하지만, 해시 조인은 Build·Probe를 각각 한 번씩 스캔하고 매칭은 해시 조회로 끝냅니다. 그래서 인덱스로 좁히기 어려운 대량 집합끼리의 조인에서 유리해집니다. 참고로 Build Input이 Hash Area에 다 담기지 못하면 두 입력을 같은 기준으로 파티션 분할해 Temp를 거치는 Grace Hash(one-pass·multi-pass) 방식으로 넘어가지만, 이때도 각 입력을

되읽는 반복 스캔이 생기는 것은 아닙니다.

나머지는 키워드에서 곧장 결론으로 건너뛴 반사적 연상입니다.

- ②: '해시=고르게 분산'이라는 반사입니다. Probe 행이 같은 함수로 자기 버킷을 찾아가는 것은 맞지만, 그 함수가 값을 흠뻑 담아도 같은 값은 같은 버킷으로 갑니다. 조인 키가 특정 값에 쏠려 있으면(skew) 그 값의 버킷만 비대해져 조회가 느려지므로, 읽기 횟수가 같아도 조회 시간이 일정하다는 결론은 성립하지 않습니다.
- ③: '조인 전에 줄 세운다=정렬'이라는 반사입니다. 양쪽을 정렬해 맞대는 것은 소트 머지 조인이고, 해시 조인은 정렬이 아니라 해시 함수로 버킷을 잡으므로 정렬 단계 자체가 없습니다. 없는 단계를 인덱스로 건너뛴다는 전제부터 성립하지 않고, 입력별 1회 읽기의 이점도 정렬과는 무관합니다.
- ④: '해시로 흠뻑 다=아무 조건이나 매칭'이라는 반사입니다. 해시 버킷은 같은 값을 한곳에 모으는 구조라 등치(=) 조인에서만 성립하며, 버킷의 키 범위를 넓게 잡아도 부등호·범위 조인에는 쓰지 못합니다 — 범위 비교를 감당하는 쪽은 정렬해 맞대는 소트 머지 조인입니다.

선택지	판정	이유
①	○	Build Input 1회·Probe Input 1회로 두 입력을 각각 한 번씩만 읽고, NL 같은 반복 스캔이 없어 대량 집합 조인에서 이점이 됩니다
②	×	Probe 행이 같은 해시 함수로 자기 버킷을 찾아가는 것은 맞지만, 같은 값은 같은 버킷으로 모이므로 조인 키가 쏠리면 그 버킷만 비대해져 느려집니다. 읽기 횟수가 같으면 시간도 일정하다는 결론이 틀렸고, '해시=균등 분포'는 반사적 연상입니다
③	×	양쪽을 정렬해 맞대는 것은 소트 머지 조인입니다. 해시 조인은 정렬을 쓰지 않고 해시 함수로 버킷을 잡으므로, 조인 키 인덱스로 정렬 순서를 얻어도 아낄 정렬 비용이 애초에 없고 입력별 1회 읽기의 이점도 정렬과 무관합니다
④	×	해시 버킷은 같은 값을 모으는 구조라 등치(=) 조인에서만 성립합니다. 버킷의 키 범위를 넓게 잡아도 한 버킷 안에서 부등호·범위 비교를 매칭할 수 없고, 범위 조인을 감당하는 쪽은 정렬해 맞대는 소트 머지 조인입니다

▪ 이 문제의 핵심

1. 두 입력을 각각 한 번씩만 읽는다. Build 1회·Probe 1회로, NL의 반복 스캔이 없어 대량 집합 조인에서 유리합니다.
2. 해시는 균등 분포를 보장하지 않는다. 같은 값은 같은 버킷으로 모여, 조인 키가 쏠리면 특정 버킷이 비대해집니다.
3. 정렬해 맞대는 것은 소트 머지 조인. 해시 조인은 정렬 단계가 없으므로 '정렬 비용 절감' 논리가 성립하지 않습니다.
4. 등치 전용. 해시 버킷은 같은 값을 모으는 구조라 부등호·범위 조인에는 쓰지 못합니다.

▪ 한 줄 정리 해시 조인은 Build·Probe를 각각 한 번씩만 읽어 대량 집합 조인에 유리한 등치 전용 기법이며, "해시니까 고르게 분산"·"조인 전에 정렬"·"범위 조인도 매칭"은 키워드에서 곧장 결론으로 건너뛴 반사적 연상입니다.

문제 12. 정답 ②

쿼리 변환은 모두가 비용 기반인 것이 아닙니다. 하는 일과 발동 방식에 따라 두 갈래로 나뉩니다.

휴리스틱(규칙 기반) 변환 : 결과가 늘 동일하고 대체로 이득 → 비용 비교 없이 규칙적으로 적용

예) 조건절 Pushing, 단순 뷰 병합, 상당수 서브쿼리 Unnesting

비용 기반 변환 : 이득이 상황에 따라 갈림 → 변환 전후 비용을 건줘 채택 여부 결정

예) 복합 뷰 병합, Join Predicate Pushdown, OR-Expansion

②는 "쿼리 변환은 옵티마이저의 최적화 → 그러니 전부 비용 기반이고 늘 이득"이라는 표면 인상에서 결론을 반사적으로 끌어냈습니다. 그러나 휴리스틱 변환은 비용을 건주지 않고 규칙적으로 적용되므로, 드물게는 변환 후가 오히려 불리한 계획이 될 수 있습니다. 복합 뷰 병합 같은 변환을 옵티마이저가 굳이 비용으로 판단하는 이유 자체가, 병합이 손해가 될 때가 있기 때문입니다. "변환하면 언제나 빨라진다"는 전제가 틀렸으므로 ②가 부적절합니다.

나머지는 모두 옳습니다.

①: 조건절 Pushing·단순 뷰 병합처럼 결과가 동일하고 대체로 이득인 변환은 비용 비교 없이 규칙적으로 적용하는 휴리스틱 변환입니다.

③: 복합 뷰 병합·Join Predicate Pushdown은 이득이 상황에 따라 갈려, 옵티마이저가 변환 전후 비용을 건줘 채택 여부를 정하는 비용 기반 변환입니다.

④: **NO_UNNEST·NO_MERGE** 같은 힌트로 개별 변환을 선택적으로 억제해, 옵티마이저의 변환 판단을 사람이 덮어쓸 수 있습니다.

선택지	판정	이유
①	○	조건절 Pushing·단순 뷰 병합처럼 결과가 동일하고 이득이 확실한 변환은 비용 비교 없이 규칙적으로 적용하는 휴리스틱 변환입니다
②	×	휴리스틱 변환은 비용을 안 견주고 적용돼 드물게 불리할 수 있고, 복합 뷰 병합을 비용으로 판단하는 이유도 손해가 날 때가 있어서입니다. '변환은 전부 비용 기반이라 늘 빨라진다'는 반사적 연상입니다
③	○	복합 뷰 병합·Join Predicate Pushdown은 이득이 갈려, 변환 전후 비용을 견줘 채택 여부를 정하는 비용 기반 변환입니다
④	○	NO_UNNEST·NO_MERGE 힌트로 개별 변환을 선택적으로 억제해 옵티마이저의 변환 판단을 사람이 덮어쓸 수 있습니다

▪ 이 문제의 핵심

1. 쿼리 변환은 휴리스틱·비용 기반 두 갈래. 전부가 비용으로 판단되는 것이 아닙니다.
2. 휴리스틱 변환은 비용 비교 없이 적용. 조건절 Pushing·단순 뷰 병합이 예이며, 드물게 불리할 수도 있습니다.
3. 비용 기반 변환은 전후 비용을 견줌. 복합 뷰 병합·Join Predicate Pushdown은 이득이 상황에 따라 갈리기 때문입니다.
4. 힌트로 개별 변환을 억제. **NO_UNNEST·NO_MERGE**로 옵티마이저의 변환 판단을 사람이 덮어쓸 수 있습니다.

▪ 한 줄 정리 쿼리 변환은 휴리스틱 변환과 비용 기반 변환으로 나뉘고 휴리스틱 변환은 비용을 견주지 않고 적용되므로, "변환은 어느 것이든 비용 기반이라 계획이 느려지는 일은 없다"는 진술은 표면 인상에서 결론을 끌어낸 반사적 연상입니다.

문제 13. 정답 ④

바인드 변수는 리터럴 값의 차이만 흡수합니다. 값이 달라도 SQL 텍스트가 그대로이므로 같은 부모 커서를 공유하게 해 주는 것이지, 텍스트 자체의 차이(공백·대소문자·주석)를 없애 주는 장치가 아닙니다. 커서 공유의 1차 관문은 여전히 SQL 텍스트가 문자 단위로 같은가이며, 이 관문은 바인드 사용 여부와 무관하게 작동합니다.

④는 "바인드를 쓰면 오라클이 공백·대소문자·주석을 정규화해 비교한다"는 정규화 가정의 오류입니다. **SELECT COUNT(*) ...:b1**과 **SELECT count(*) ...:b1**은 바인드를 똑같이 쓰더라도 **COUNT/count**의 대소문자와 공백이 달라 서로 다른 SQL_ID로 갈리고, 각각 별도 부모 커서로 하드파싱됩니다. 오라클은 텍스트를 정규화하지 않으므로, 바인드는 값의 다양성만 하나로 묶을 뿐 텍스트 변형까지 묶어 주지 못합니다. 그래서 부적절한 진술은 ④입니다.

①②③은 각각 바인드로 값만 다른 실행이 부모 커서를 공유해 소프트파싱으로 처리된다는 점(①), 실행 중 커서의 핀과 미사용 커서의 LRU 에이징(②), 통계 재수집·DDL에 의한 커서 무효화(③)를 옳게 서술합니다.

선택지	판정	이유
①	○	텍스트가 같고 값만 :b1·:b2로 다른 실행은 같은 부모 커서를 공유해, 최초 하드파싱 뒤 소프트파싱으로 재사용되므로 library cache: mutex 경합이 줄어듭니다
②	○	실행 중 커서는 핀되어 에이징에서 빠지고, 사용이 끝난 커서는 메모리 압박 시 LRU로 밀려나며 필요 시 재파싱됩니다
③	○	대상 객체의 통계 재수집·DDL은 관련 커서를 무효화하므로, 다음 실행에서 재파싱·재최적화가 일어납니다
④	×	바인드는 리터럴 값 차이만 흡수할 뿐 텍스트를 정규화하지 않습니다. COUNT/count ·공백이 다르면 바인드를 써도 다른 SQL_ID로 하드파싱됩니다

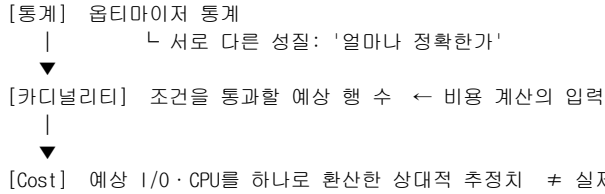
▪ 이 문제의 핵심

1. 바인드 변수는 값의 다양성만 하나로 묶습니다. 텍스트가 같고 값만 다른 실행이 같은 부모 커서를 공유하게 해, 하드파싱을 소프트파싱으로 바꿉니다.
2. 커서 공유의 1차 관문은 텍스트 완전 일치입니다. 바인드를 써도 공백·대소문자·주석이 다르면 다른 SQL_ID로 갈립니다 — 오라클은 텍스트를 정규화하지 않습니다.
3. 핀·에이징은 라이브러리 캐시의 수명 관리입니다. 실행 중 커서는 핀되어 보호되고, 미사용 커서는 메모리 압박 시 LRU로 에이징됩니다.
4. 무효화는 커서 재파싱의 방아쇠입니다. 통계 재수집·DDL 등 대상 객체의 변화가 관련 커서를 무효화해 다음 실행에서 재최적화를 부릅니다.

▪ 한 줄 정리 바인드 변수는 리터럴 값 차이만 흡수해 커서를 공유시킬 뿐 텍스트를 정규화하지 않으므로, "바인드면 공백·대소문자·주석까지 묶어 공유한다"는 ④는 정규화 가정의 오류입니다.

문제 14. 정답 ②

CBO의 비용 산정에는 서로 독립인 측들이 얹혀 있습니다. ②는 그 관계를 뒤섞지 않고 바르게 진술합니다.



Cost는 옵티마이저가 여러 후보 계획 중 어느 쪽이 더 싼지 상대 비교하려고 만든 내부 수치입니다. 통계에서 추정된 카디널리티가 입력으로 쓰이지만, Cost 값 자체는 초 단위 실제 시간과 일대일로 맞물리지 않습니다 — 캐시 적중·하드웨어·동시 부하 같은 실행 시점 요인이 실제 시간을 좌우하기 때문입니다. ②가 이 세 측(통계 정확도 → 카디널리티 → 상대 비용)의 관계를 정확히 짚어 줍니다.

나머지는 독립인 두 측을 인과로 엮은 별개 측 혼동입니다.

①: **optimizer_index_cost_adj**는 인덱스 액세스 비용에 공급하는 가중치(산정 측)일 뿐, 인덱스의 물리적 효율(클러스터링 팩터)을 바꾸지 않습니다. 가중치를 낮추면 산정 비용만 내려가지, 인덱스가 실제로 더 좋아지는 것이 아닙니다.

③: 카디널리티(예상 행 수)와 Cost(자원 환산 추정치)는 다른 측입니다. 행 수가 정확해도 Cost는 상대 비교용 수치라 실제 수행 시간과 일치하지 않습니다.

④: I/O 비용과 CPU 비용은 각기 다른 자원을 재는 독립 성분입니다. 대량 정렬처럼 CPU는 크지만 메모리 처리로 I/O는 작은 계획도 있어, 한쪽만 보고 총비용의 우열을 단정할 수 없습니다.

선택지	판정	이유
①	×	optimizer_index_cost_adj 는 인덱스 비용에 공급하는 가중치일 뿐, 클러스터링 팩터 같은 물리적 효율을 바꾸지 않습니다. 산정 측과 물리 효율 측을 엮은 혼동입니다
②	○	Cost는 예상 I/O · CPU를 환산한 상대적 추정치로, 통계발 카디널리티에서 출발하지만 실제 경과 시간과 일대일 대응하지 않습니다
③	×	카디널리티(예상 행 수)와 Cost(자원 추정치)는 다른 측입니다. 행 수가 맞아도 Cost는 상대 비교용이라 실제 시간과 일치하지 않습니다
④	×	I/O 비용과 CPU 비용은 독립 성분입니다. CPU는 크되 I/O는 작은 계획도 있어 한쪽만으로 총비용 우열을 가릴 수 없습니다

▪ 이 문제의 핵심

1. Cost는 상대 비교용 추정치. 후보 계획의 우열을 가리려 예상 I/O · CPU를 하나로 환산한 값이며, 실제 초 단위 시간과 일대일 대응이 아닙니다.
2. 통계 → 카디널리티 → 비용의 흐름. 통계 정확도가 카디널리티를, 카디널리티가 비용을 좌우하되 셋은 서로 다른 측입니다.
3. **optimizer_index_cost_adj**는 가중치 측. 산정 비용을 조정할 뿐 인덱스의 물리적 효율을 바꾸지 않습니다.
4. I/O 비용 ≠ CPU 비용. 독립 성분이라 한쪽만으로 총비용의 우열을 단정할 수 없습니다.

▪ 한 줄 정리 CBO의 Cost는 예상 I/O · CPU를 환산한 상대적 추정치로 통계발 카디널리티에서 출발하되 실제 시간과 일대일 대응하지 않는다는 ②가 옳고, 인덱스 가중치를 물리 효율로 · 카디널리티를 실제 시간으로 · CPU 비용을 I/O 비용으로 잇는 나머지는 독립인 측을 인과로 엮은 별개 측 혼동입니다.

문제 15. 정답 ③

TRUNCATE PARTITION은 행 단위로 지우는 DML이 아니라, 파티션 세그먼트의 저장 영역을 통째로 해제하고 HWM을 리셋하는 DDL입니다. DDL이므로 행별 Undo를 쌓지 않고, 실행 시점에 암묵적으로 커밋되어 트랜잭션의 일부로 ROLLBACK되지 않습니다.

DELETE FROM ... WHERE (행 단위) : DML · 행마다 Undo · Redo 다량 · ROLLBACK 가능 · HWM 유지
 TRUNCATE PARTITION (세그먼트 단위) : DDL · 저장영역 해제 · HWM 리셋 · 암묵적 COMMIT · ROLLBACK 불가
 ⇒ 대량 삭제에서 Undo · Redo가 훨씬 적고 빠르다

③은 두 성질을 정반대로 뒤집었습니다. 첫째, TRUNCATE는 "행을 한 건씩 지우며 행마다 Undo를 남기는" 방식이 아니라 세그먼트를 통째로 비우므로 Undo가 DELETE보다 훨씬 적습니다. 둘째, TRUNCATE는 DDL이라 암묵적 커밋되어 ROLLBACK으로 되돌릴 수 없습니다. ③은 "행 단위 · Undo 다량 · 롤백 가능"이라고 서술해 방향과 되돌림 가능성을 모두 반대로 말했으므로, 부적절한 것은 ③입니다.

①②④는 각각 APPEND의 Direct Path Insert 동작(①), nologging + Direct Path의 최소 로깅과 복구 위험(②), 병렬 DML의 활성화 조건과 같은 트랜잭션 내 재접근 제약(④)을 옳게 서술합니다.

선택지	판정	이유
①	○	APPEND는 Direct Path Insert로 HWM 위에 새 블록을 포맷해 써 넣고 버퍼 캐시를 우회하므로, 재사용 블록 탐색과 버퍼 경합을 줄여 대량 적재에 유리합니다
②	○	nologging + Direct Path Insert는 redo를 최소화해 로그 부하를 줄이지만, 그 데이터는 미디어 복구 대상에서 빠져 복구 불가 위험을 안습니다
③	×	TRUNCATE PARTITION은 세그먼트를 통째로 비우는 DDL이라 Undo가 DELETE보다 적고 암묵적 커밋되어 ROLLBACK되지 않습니다 — 방향·되돌림이 반대입니다
④	○	병렬 DML은 세션 활성화가 전제이며, 변경한 테이블은 같은 트랜잭션 안에서 재조회·재변경할 수 없어 커밋 뒤에 접근해야 합니다

▪ 이 문제의 핵심

1. APPEND는 Direct Path Insert입니다. HWM 위에 새 블록을 직접 써 넣고 버퍼 캐시를 우회해 대량 적재의 재사용 블록 탐색·버퍼 경합을 없앱니다.
2. nologging + Direct Path는 redo 최소화입니다. 로그 부하는 줄이지만 미디어 복구에서 그 데이터가 빠지는 위험을 감수해야 합니다.
3. TRUNCATE PARTITION은 DDL입니다. 세그먼트를 통째로 해제해 Undo·Redo가 DELETE보다 훨씬 적고, 암묵적 커밋되어 ROLLBACK되지 않습니다.
4. 병렬 DML은 세션 활성화가 전제이며, 변경한 객체는 같은 트랜잭션 안에서 다시 접근할 수 없어 커밋 뒤에 읽어야 합니다.

▪ 한 줄 정리 TRUNCATE PARTITION은 세그먼트를 통째로 비우는 DDL이라 Undo가 적고 롤백되지 않는데, ③은 "행 단위로 Undo 남기며 롤백 가능"이라 정반대로 서술해 부적절합니다.

문제 16. 정답 ③

복합 파티션에서 프루닝은 RANGE 축(제공일자) 과 HASH 축(학교코드) 을 따로 판정합니다. RANGE 축은 경계값과 대조해, HASH 축은 정확한 값(등치) 의 해시 결과로 서브파티션을 좁힙니다. 두 축 모두 키에 가공이 없어야 대조·해시가 가능하고, 서브파티션 프루닝은 파티션 키인 학교코드에 대해서만 걸립니다.

[조회별 접근 범위 판정]

- ① 제공일자 ≥ 2026-07-01 → RANGE: p_2026q3 · p_max / HASH: 학교코드 조건 없음 → 서브 8개 전부
- ② 학교코드=30012 & 식단구분='중식' → HASH: 서브 1개(학교코드 등치) / 식단구분은 파티션 키 아님 → 추가 프루닝 없음
- ③ 제공일자=2026-05-10 & 학교코드=30012 → RANGE: p_2026q2 1개 / HASH: 서브 1개 → 최종 단일 서브파티션 ○
- ④ TO_CHAR(학교코드) LIKE '300%' → 키 가공(형변환+LIKE) → 해시 대조 불가 → 서브 전부 스캔

③은 두 축을 모두 등치로 걸었습니다. 제공일자 등치로 RANGE가 p_2026q2 한 파티션으로, 학교코드 등치로 HASH가 그 안 서브파티션 1개로 좁혀져 파티션 1개 × 서브파티션 1개 = 최종 단일 서브파티션만 읽습니다. 두 축 모두 키에 가공이 없어 대조·해시가 성립하므로, 설명대로 프루닝이 작동하는 것은 ③입니다.

나머지는 모두 어긋납니다. ①은 학교코드 조건이 없는데 서브파티션 1개로 좁힌다고 했으나 해시 축을 좁힐 등치 값이 없어 서브 8개를 전부 읽습니다. ②는 식단구분을 마치 파티션 키처럼 다루 "한 번 더 프루닝"이 걸린다고 했지만 식단구분은 파티션 키가 아니라 일반 필터일 뿐 세그먼트를 줄이지 못합니다. ④는 TO_CHAR(학교코드) LIKE '300%'로 키를 형변환·가공해, 앞 3자리가 같아도 해시가 흩어져 서브파티션을 특정할 수 없으므로 전 서브파티션을 스캔합니다.

선택지	판정	이유
①	×	제공일자 범위로 RANGE는 p_2026q3 · p_max로 좁혀도, 학교코드 등치가 없어 해시 축은 못 좁힙니다 — 서브파티션 8개를 전부 읽습니다
②	×	식단구분은 파티션 키가 아니라 일반 필터라 서브파티션을 더 줄이지 못합니다. "한 번 더 프루닝으로 절반"은 성립하지 않습니다
③	○	제공일자·학교코드 둘 다 등치라 RANGE로 p_2026q2 1개, HASH로 서브 1개까지 좁혀 최종 단일 서브파티션만 읽습니다
④	×	TO_CHAR(학교코드) LIKE '300%'는 키를 형변환·가공한 것이라 해시 대조가 불가능해 전 서브파티션을 스캔합니다. "앞 3자리로 겨냥"은 프루닝을 보장하지 못하는 부분진실입니다

▪ 이 문제의 핵심

1. 복합 파티션 프루닝은 RANGE 축·HASH 축을 따로 판정합니다. 각 축의 키가 가공 없이 걸려야 대조·해시가 가능합니다.

2. HASH 서브파티션 프루닝은 파티션 키(학교코드)의 등치 조건에서만 걸립니다. 파티션 키가 아닌 식단구분 같은 컬럼은 세그먼트를 줄이지 못합니다.
3. 두 축을 모두 등치로 걸면(③) 파티션 1개 × 서브파티션 1개로 최종 단일 세그먼트까지 좁혀집니다.
4. 키를 형변환·가공(TO_CHAR·LIKE·SUBSTR) 하면 해시 대조가 불가능해 서브파티션을 전부 스캔합니다 — "앞자리로 겨냥"이라는 부분진실이 함정입니다.

▪ 한 줄 정리 복합 파티션 프루닝은 제공일자·학교코드를 각각 등치로 걸 때 파티션 1개 × 서브파티션 1개로 좁혀지므로 두 축을 모두 등치로 건 ③이 적절하고, 조건 없는 해시 축·비키 필터·키 형변환인 ①②④는 프루닝이 작동하지 않습니다.

문제 17. 정답 ①

분배 방식은 어느 입력이 **PX SEND**를 거치고 그 SEND의 종류가 무엇인지로 읽습니다.

Id 6	PX SEND BROADCAST :TQ10000 (Q1,00 → 모든 서버)	소형 지원유형을 전 서버에 복제
Id 7	PX BLOCK ITERATOR (Q1,00)	지원유형 블록 스캔
Id 8	TABLE ACCESS FULL 지원유형 (약 700건)	복제 대상 - 작다
Id 9	PX BLOCK ITERATOR (Q1,01)	지급신청을 각 서버가 자기 블록만
Id 10	TABLE ACCESS FULL 지급신청 (약 3.2억 건)	재분배 SEND 없음

플랜에서 조인 입력 중 **PX SEND**를 거치는 쪽은 지원유형(Id 8) 하나뿐이고, 그 SEND는 **PX SEND BROADCAST**(Id 6)입니다. 즉 소형 지원유형을 모든 병렬 서버에 복제합니다.

대형 지급신청(Id 10)은 위에 아무 SEND가 없고 **PX BLOCK ITERATOR**(Id 9)로 각 서버가 자기 블록 범위만 읽습니다. 재분배되지 않습니다.

이 조합이 broadcast 분배입니다. 3억 건이 넘는 지급신청을 해시로 재분배하면 그 큰 집합 전체가 서버 사이클을 오가야 하지만, 700건짜리 지원유형을 복제하면 트래픽이 미미합니다. 그래서 대형×소형 조인에서 옵티마이저가 소형 broadcast를 고른 것입니다.

- | | | |
|---------------|--|-------------------------|
| ① broadcast | : 소형 쪽에 PX SEND BROADCAST(Id 6), 대형은 SEND 없음 | → Id 6·9·10과 일치 |
| ② hash-hash | : 양쪽 위에 각각 PX SEND HASH | → 대형 지급신청엔 SEND 없음 |
| ③ full PWJ | : 두 입력이 같은 PX PARTITION 아래, SEND 없음 | → 소형에 BROADCAST SEND 있음 |
| ④ partial PWJ | : 소형에 PX SEND PARTITION (KEY) | → SEND 종류가 BROADCAST |

유일한 조인 입력 SEND가 **PX SEND BROADCAST** 하나라는 사실이 hash-hash·full PWJ·partial PWJ를 한꺼번에 배제합니다. 따라서 ①이 옳습니다.

선택지	판정	이유
①	○	조인 입력 중 SEND를 거치는 쪽은 소형 지원유형(Id 8)뿐이고 그 SEND가 PX SEND BROADCAST (Id 6), 대형 지급신청(Id 10)은 SEND 없이 PX BLOCK ITERATOR 로 로컬 스캔 — 대형×소형에서 소형을 복제하는 broadcast 분배입니다
②	×	hash-hash면 두 입력 위에 각각 PX SEND HASH 가 있어야 하는데, 대형 지급신청(Id 10) 위엔 SEND가 없이 Id 9 PX BLOCK ITERATOR 뿐이고 소형 쪽 Id 6은 SEND HASH가 아니라 SEND BROADCAST입니다. 조인 입력 테이블 큐드 :TQ10000 하나뿐이라 '두 갈래'가 성립하지 않는, 분배 방식을 broadcast에서 hash-hash로 뒤바꾼 진술입니다
③	×	full 파티션 와이즈면 두 입력이 같은 PX PARTITION 아래에서 SEND 없이 읽혀야 하는데, 지원유형은 PX SEND BROADCAST (Id 6)로 복제되고 있고 지급신청은 지원유형 기준 파티션도 아닙니다. Pstart·Pstop이 빈 것은 파티션 액세스가 없다는 뜻이지 파티션 쌍을 맡았다는 표시가 아니므로, broadcast를 PWJ로 뒤바꾼 값스왑입니다
④	×	partial 파티션 와이즈면 재분배되는 소형 쪽에 PX SEND PARTITION (KEY) 가 있어야 하는데, Id 6의 SEND는 BROADCAST 이고 Id 5 PX RECEIVE 는 700건 전량을 받습니다. Id 9 PX BLOCK ITERATOR 도 파티션 단위가 아니라 블록 범위 단위 분할이라, SEND 종류를 BROADCAST에서 PARTITION (KEY)로 뒤바꾼 진술입니다

▪ 이 문제의 핵심

1. 분배 방식은 조인 입력의 **PX SEND** 종류로 판별합니다. 여기서 조인 입력 SEND는 **PX SEND BROADCAST**(Id 6) 하나뿐입니다.
2. broadcast는 대형×소형에서 소형을 전 서버에 복제하는 방식입니다. 700건짜리 지원유형을 복제하고, 3.2억 건 지급신청은 재분배 없이 각 서버가 블록 단위로 읽습니다.
3. 대형 지급신청 위에 SEND가 없다는 것이 핵심 단서입니다. Id 9 **PX BLOCK ITERATOR**로 로컬 스캔만 하므로 큰 집합이 서버 사이클을 오가지 않습니다.
4. hash-hash·full PWJ·partial PWJ는 각각 **PX SEND HASH**·같은 PX PARTITION 아래·**PX SEND PARTITION (KEY)**가 있어야 성립합니다. 이 플랜의 SEND는 BROADCAST뿐이라 셋 다 배제됩니다 — 분배 방식을 뒤바꾼 값스왑 오답입니다.

- 한 줄 정리 조인 입력 중 소형 지원유형(Id 8)만 **PX SEND BROADCAST**(Id 6)로 전 서버에 복제되고 대형 지급신청(Id 10)은 SEND 없이 **PX BLOCK ITERATOR**로 로컬 스캔하므로, 대형을 재분배하는 대신 소형을 복제한 broadcast 분배다.

문제 18. 정답 ④

인덱스가 지우는 것은 그 인덱스의 정렬 순서와 일치하는 하나의 소트입니다. 인덱스 하나는 한 가지 정렬 순서만 제공합니다.

인덱스 1개 = 정렬 순서 1가지 제공

분석함수 A: PARTITION BY 부서, ORDER BY 급여 ← 이 순서와 맞는 인덱스면 A의 WINDOW SORT 생략

분석함수 B: PARTITION BY 지역, ORDER BY 입사일 ← 순서가 달라 같은 인덱스로 대체 불가 → 별도 정렬

⇒ 여러 분석함수의 정렬 요구가 서로 다르면, 인덱스로 지워지는 건 그중 '하나'뿐

④는 "인덱스로 분석함수 정렬이 대체된다"는 부분적 사실을, "인덱스만 있으면 여러 분석함수의 WINDOW SORT가 남김없이 사라진다"는 전체 등식으로 부풀렸습니다. 서로 다른 **PARTITION BY·ORDER BY**를 쓰는 분석함수가 여럿이면 요구하는 정렬 순서도 제각각이라, 한 인덱스의 순서와 맞는 하나의 WINDOW SORT만 생략될 수 있고 나머지는 여전히 정렬해야 합니다. 부분 조건을 전체로 넓힌 하위항목 전체화라 ④가 부적절합니다.

나머지는 모두 옳습니다.

①: 분석함수는 **PARTITION BY·ORDER BY** 순서로 행을 정렬해야 윈도우를 적용하므로, WINDOW SORT가 나타나는 것이 일반적입니다.

②: 분석함수의 정렬 순서와 맞는 인덱스로 정렬된 순서를 공급받으면 WINDOW NOSORT로 바뀌어 정렬 단계가 생략됩니다.

③: 그룹 집계와 분석함수가 요구하는 정렬 순서가 서로 다르면, SORT GROUP BY와 별개로 WINDOW SORT가 추가로 필요할 수 있습니다.

선택지	판정	이유
①	○	분석함수는 PARTITION BY·ORDER BY 순서로 행을 정렬해야 윈도우를 적용하므로 WINDOW SORT가 나타나는 것이 일반적입니다
②	○	그 정렬 순서와 맞는 인덱스로 정렬된 순서를 공급받으면 WINDOW NOSORT로 바뀌어 정렬 단계가 생략됩니다
③	○	그룹 집계와 분석함수의 정렬 순서가 다르면 SORT GROUP BY와 별개로 WINDOW SORT가 추가로 필요할 수 있습니다
④	×	인덱스는 한 가지 정렬 순서만 주므로 서로 다른 순서를 쓰는 여러 분석함수 중 하나의 WINDOW SORT만 지워질 수 있습니다. '인덱스만 있으면 전부 사라진다'는 부분을 전체로 부풀린 하위항목 전체화입니다

▪ 이 문제의 핵심

1. 분석함수는 WINDOW SORT를 부른다. **PARTITION BY·ORDER BY** 순서로 행을 정렬해야 윈도우를 적용하기 때문입니다.
2. 정렬 순서가 맞는 인덱스면 WINDOW NOSORT. 정렬된 순서로 공급받아 정렬 단계를 생략합니다.
3. 인덱스 하나는 정렬 순서 하나만 제공. 서로 다른 순서를 쓰는 여러 분석함수 중 지워지는 건 그와 맞는 하나뿐입니다.
4. 그룹 집계 정렬과 분석함수 정렬은 별개일 수 있다. 순서가 다르면 WINDOW SORT가 추가로 필요합니다.

▪ 한 줄 정리 인덱스는 한 가지 정렬 순서만 주므로 서로 다른 순서를 쓰는 여러 분석함수 중 하나의 WINDOW SORT만 생략될 수 있는데, "인덱스만 갖춰 두면 여러 분석함수의 WINDOW SORT가 없이 처리된다"는 진술은 부분 조건을 전체 등식으로 부풀린 하위항목 전체화입니다.

문제 19. 정답 ④

SELECT ... FOR UPDATE는 단순 조회가 아니라 현재 행에 잠금을 거는 문장입니다. 잠금은 언제나 최신 커밋된 행 버전에 걸려야 하므로, FOR UPDATE는 일반 SELECT처럼 문장 시작 스냅샷(CR 버전)을 읽는 게 아니라 잠금을 획득하는 시점에 커밋돼 있는 현재값을 읽습니다(쓰기 일관성, write consistency).

[TX2의 SELECT ... FOR UPDATE 값 계산]

t2 FOR UPDATE가 가·나·다에 잠금 시도

– 가 : 잠금 가능

– 나·다 : TX1이 t1에 잠가 둬(미커밋) → TX2 대기(블로킹)

t3 TX1 COMMIT → 나=250000, 다=380000 확정, 잠금 해제

t4 TX2 재개 → 재시작(restart)하여 '해제 시점의 최신 커밋값'을 잠그고 읽음

가 = 100000 (변경 없음)

나 = 250000 (t3 커밋된 최신값)

다 = 380000 (t3 커밋된 최신값)

SUM = 100000 + 250000 + 380000 = 730000

핵심은 FOR UPDATE가 대기 후 재시작해 커밋된 최신값을 잠그고 읽는다는 점입니다. 만약 같은 상황에서 일반 SELECT였다면 문장 시작 시점 스냅샷을 Undo로 CR 재구성해 600,000을 읽었겠지만, FOR UPDATE는 잠글 대상의 현재값을 봐야 하므로 t3에 커밋된 250,000·380,000을 반영합니다. 따라서 SUM = 730,000, 정답은 ④입니다.

①②③은 스냅샷값과 부분 갱신값을 섞어 만든 값스왑 오답입니다. ①은 일반 SELECT라면 나올 문장 시작 스냅샷 600,000, ②는 나만 갱신값(250000)으로 섞은 650,000, ③은 다만 갱신값(380000)으로 섞은 680,000입니다.

선택지	판정	이유
①	×	100000+200000+300000. 일반 SELECT의 문장 시작 스냅샷 값이며, 현재값을 잠그는 FOR UPDATE에는 맞지 않습니다
②	×	100000+250000+300000. 나만 커밋된 최신값(250000), 다는 원래값으로 섞은 값스왑입니다
③	×	100000+200000+380000. 다만 커밋된 최신값(380000), 나는 원래값으로 섞은 값스왑입니다
④	○	FOR UPDATE는 대기 후 재시작해 t3에 커밋된 최신값(나 250000·다 380000)을 잠그고 읽으므로 SUM=730000입니다

■ 이 문제의 핵심

1. SELECT ... FOR UPDATE는 현재 행을 잠그는 문장입니다. 잠금은 최신 커밋 버전에 걸려야 하므로 문장 시작 스냅샷이 아니라 커밋된 현재값을 읽습니다.
2. 블로킹 후 재시작(restart) 합니다. 앞선 트랜잭션이 커밋해 잠금이 풀리면, FOR UPDATE는 해제 시점의 최신 커밋값을 다시 읽어 잠급니다.
3. 일반 SELECT와 다른 각도입니다. 같은 타임라인에서 순수 SELECT라면 CR 재구성으로 600,000을 읽지만, FOR UPDATE는 730,000을 읽습니다.
4. Undo는 CR 재구성의 재료입니다. 일반 SELECT의 문장 수준 읽기 일관성은 Undo로 과거 버전을 되감아 얻지만, FOR UPDATE는 되감지 않고 현재값을 잠급니다.

■ 한 줄 정리 FOR UPDATE는 현재 행을 잠그느라 대기 후 재시작해 t3에 커밋된 최신값(250000·380000)을 읽으므로 SUM=730,000이 되고, 문장 시작 스냅샷(600,000)이나 부분 조합(650,000·680,000)은 오답입니다.

문제 20. 정답 ②

blocking_session_id는 나를 막고 있는 세션을 가리킵니다. 값이 0이면 그 세션은 아무에게도 막혀 있지 않은 블로킹 체인의 선두(head)입니다. 표를 그대로 읽으면 53(0) ← 61(53) ← 74(61)로 이어지는 선형 체인이며, 순환(cycle)이 아닙니다.

[블로킹 체인 판독]

53 : blocking_session_id=0, running → 선두(막는 자, 스스로는 대기 안 함)
 61 : blocking_session_id=53, LCK_M_X 대기 → 53에 막힘
 74 : blocking_session_id=61, LCK_M_S 대기 → 61에 막힘 (53이 아니라 61)

방향: 53 —막음—▶ 61 —막음—▶ 74 (선형, 순환 아님)

해소: 53 커밋/롤백 → 61이 X 획득 → 61 커밋/롤백 → 74가 S 획득

②는 이 체인을 정확히 읽었습니다. 대기를 풀려면 선두 53의 트랜잭션을 끝내야 하고, 그러면 61이 잠금을 얻어 진행한 뒤 74가 이어서 풀립니다(61 → 74 순). 그래서 적절한 해석은 ②입니다.

나머지는 어긋납니다. ①은 "세 세션이 모두 잠금 대기이니 교착"이라 했는데, 이는 블로킹에도 공통으로 나타나는 단서(LCK 대기)를 교착의 근거로 착각한 것입니다. 교착은 대기가 순환을 이뤄야 성립하는데, 이 표는 53이 아무에게도 안 막힌 선형 체인이라 교착 모니터가 개입하지 않습니다. ③은 74의 blocking_session_id가 61인데도 "53 때문"이라 자원 제공자를 바꿔 읽었고, 큐 앞에 배타 요청이 있으면 S도 대기하므로 "S는 폭넓게 호환"이라는 전제가 이 상황에서 성립하지 않습니다. ④는 53이 순환에 얽혀 74에 막힌다고 했으나 53의 blocking_session_id는 0(막힌 것 없음)이며, wait_time이 가장 큰 세션도 53(0ms)이 아니라 61(18000ms)입니다.

선택지	판정	이유
①	×	잠금 대기(LCK_M_*)는 블로킹에도 공통인 단서라 교착의 근거가 못 됩니다. 53이 아무에게도 안 막힌 선형 체인이라 순환이 없어 교착이 아닙니다
②	○	53(blocking_session_id=0)이 선두이고 61←53·74←61이므로, 53을 끝내면 61→74 순으로 대기가 풀립니다. 표의 세 행 그대로입니다
③	×	74의 blocking_session_id는 61입니다. 큐 앞에 X 요청이 있으면 S도 대기하므로, 61이 풀리면 74가 진행합니다 — "53 때문에 계속 막힘"이 아닙니다
④	×	53의 blocking_session_id는 0(막힌 것 없음)이라 순환이 없습니다. wait_time 최대도 53(0ms)이 아니라 61(18000ms)입니다

▪ 이 문제의 핵심

1. blocking_session_id=0 세션이 블로킹 체인의 선두입니다. 아무에게도 막히지 않은 그 세션의 트랜잭션을 끝내야 대기가 풀립니다.
2. 선형 체인은 교착이 아닙니다. 교착은 대기가 순환을 이뤄야 성립하며, 그때만 교착 모니터가 희생자를 롤백합니다.
3. "모두 잠금 대기"는 판별 근거가 못 됩니다. 블로킹에도 나타나는 공통 단서라, 그것만으로 교착을 단정하면 오독입니다.
4. 대기는 선두→후미 순으로 풀립니다. 53을 끝내면 61이 X를 얻고, 61이 끝나면 74가 S를 얻습니다 — blocking_session_id가 가리키는 방향 그대로입니다.

▪ 한 줄 정리 표는 53←61←74의 선형 블로킹 체인이라 선두 53을 끝내면 61→74 순으로 풀리며, "모두 잠금 대기이니 교착"(①)은 블로킹에도 공통인 단서를 교착 근거로 착각한 오독이므로 ②가 적절합니다.